



Faculté des Sciences et Techniques de Tanger

Université Abdelmalek Essaadi

Master's End-of-Study Project (PFE)
Master in **[TODO: your master programme name]**

Learning Offensive Navigation Policies on Heterogeneous Attack Graphs

A GNN Approach to Sequential Privilege Escalation
in Active Directory

Author:
Nizar Senbati

Supervisor:
[TODO: Supervisor name & title]

Academic Year 2025 – 2026

Acknowledgements

First and foremost, I would like to express my deepest gratitude to my academic supervisor GHADI Abderahim, for his unwavering guidance, patience, and invaluable feedback throughout this academic year.

I am equally grateful to my professional supervisor, ZAABOUL Jawad, and the entire team at NearSecure. Thank you for providing an environment that fosters innovation, for trusting me to pursue the intersection of offensive security and Deep Learning, and for the technical mentorship that bridged the gap between academic theory and real-world application.

A massive thank you is owed to the broader cybersecurity and open-source communities. The development of the GNN-AD-Navigator would not have been possible without the tireless work of the developers who build and maintain the tools we rely on. Specifically, I would like to acknowledge:

SpecterOps, for creating and open-sourcing BloodHound, which provided the foundational graph topology and data schema necessary for my model's ingestion pipeline.

Oliver Lyak ly4k, for developing Certipy, an essential tool that allowed me to accurately map complex Active Directory Certificate Services ADCS attack paths into my heterogeneous graph.

Mayfly Orange Cyberdefense, for creating GOAD Game of Active Directory. The immense complexity of this lab environment served as the perfect training ground for the neural network, allowing it to learn and discover emergent tactical behaviors.

I also want to thank Hack The Box Academy for their student program, which granted me accessible access to the inlanefreight Active Directory environment. This provided the critical cross-environment validation required to prove the model's efficacy outside of its training data.

Finally, a personal thank you to my father and family for the ongoing support, advice and guidance during the long hours spent debugging and studying. To everyone who supported me through this Master's journey: thank you.

Abstract

Active Directory is a directory service so deeply embedded in corporate infrastructure that misconfigurations, no matter how small, can expose entire organisations to privilege escalation attacks. Tools like BloodHound are critical for both defenders and red teams: they map the environment as a graph and let operators query attack paths between any two objects. BloodHound’s built-in pathfinding, however, runs plain Dijkstra — it returns the shortest path, not necessarily the most operationally sound one, and it cannot learn from prior engagements.

This thesis asks whether a Graph Neural Network can learn a navigation policy from labelled attack traces and generalise it to entirely unseen enterprise environments. A Heterogeneous Graph Transformer is trained on a multi-domain lab environment and evaluated on a 4 000-node enterprise network it has never seen. The model recovers the expert-documented Domain Admin attack path without retraining, establishing cross-environment generalisation as a proof of concept. Identified architectural boundaries and a forward-looking hypothesis for a second-generation tool are discussed.

Keywords: Graph Neural Network, Heterogeneous Graph Transformer, Active Directory, Privilege Escalation, Attack Graph, Beam Search, Offensive Security, Cross-Environment Generalisation, Active Directory Certification Services.

Résumé

Active Directory est un service d'annuaire si profondément intégré dans l'infrastructure des entreprises que la moindre erreur de configuration peut exposer une organisation entière à des attaques de privilege escalation. Des outils comme BloodHound sont essentiels à la fois pour les défenseurs et les red teams : ils cartographient l'environnement sous forme de graphe et permettent aux opérateurs de rechercher des chemins d'attaque entre deux objets. Cependant, le moteur de recherche de BloodHound utilise le simple algorithme de Dijkstra renvoie le chemin le plus court, et non le plus pertinent d'un point de vue opérationnel.

Cette thèse cherche à savoir si un Graph Neural Network peut apprendre une politique de navigation à partir de traces d'attaques étiquetées, et la généraliser à des environnements d'entreprise totalement inconnus. Un Heterogeneous Graph Transformer est entraîné sur un environnement de laboratoire multi-domaines, puis évalué sur un réseau d'entreprise de 4 000 nœuds qu'il n'a jamais vu auparavant. Le modèle parvient à retrouver le chemin d'attaque Domain Admin documenté par des experts, et ce sans réentraînement, validant ainsi la généralisation cross-environment comme un véritable Proof of Concept. Enfin, les limites architecturales identifiées ainsi que des pistes d'évolution pour un outil de seconde génération sont discutées.

Mots-clés : Réseau de neurones sur graphe, Transformateur de graphe hétérogène, Active Directory, Escalade de privilèges, Graphe d'attaque, Recherche en faisceau, Sécurité offensive, Généralisation inter-environnements.

Contents

Acknowledgements	i
Abstract	ii
Résumé	iii
List of Abbreviations	ix
General Introduction	1
1 Background and Related Work	4
1.1 Active Directory: Architecture and Security Model	4
1.1.1 Core Objects and Structure	4
1.1.2 Authentication Protocols	5
1.1.3 Trust Relationships	8
1.2 Privilege Escalation in AD Environments	9
1.2.1 Offensive ACL Abuse	9
1.2.2 DCSync Attack	10
1.2.3 ADCS Attack Paths	10
1.2.4 SID History Abuse and RBCD	11
1.3 Existing Tools and Their Limitations	11
1.3.1 BloodHound and SharpHound	11
1.3.2 Certipy	12
1.3.3 bloodhound-python	12
1.3.4 Other Automated Attack Path Tools	12
1.4 Graph Neural Networks	13
1.4.1 Message Passing and Neighbourhood Aggregation	13
1.4.2 Heterogeneous Graph Transformer (HGT)	14
1.4.3 PyTorch Geometric and HeteroData	15
1.5 Summary and Positioning	16
2 System Architecture and Design	18
2.1 System Overview	18
2.2 Data Collection and Preprocessing	19
2.2.1 clean_bloodhound.py: Filtering and Multi-Domain Accumulation	20

2.2.2	<code>merger.py</code> : Normalisation and Merging	21
2.2.3	<code>stitcher.py</code> : ADCS Injection and Domain Detection	22
2.2.4	Training Environment: GOAD	22
2.2.5	Feature engineering	23
2.3	Navigation Model	25
2.3.1	HGT Encoder	25
2.3.2	Policy Head	25
2.3.3	Loss Function and Step Cost	26
2.4	Beam Search Navigation	27
2.4.1	Beam Search Navigation	27
2.4.2	Audit Module	29
2.5	Validation Module & Logging	30
3	Implementation	31
3.1	Technical Stack	31
3.2	Repository Structure	32
3.3	Data Pipeline	32
3.3.1	Script Responsibilities and Data Flow	32
3.3.2	SharpHound Version Normalisation	33
3.4	Training	33
3.4.1	Training Data: GOAD	33
3.4.2	Kaggle Training Notebook	33
3.4.3	Model Checkpoints	33
3.5	Graphical & Command Line Interface	33
3.5.1	CLI	33
3.5.2	GUI	33
4	Experiments and Results	34
4.1	Experimental Setup	34
4.1.1	Test Environment: INLANEFREIGHT	34
4.1.2	Diagnostic Environment: GOAD	34
4.1.3	Evaluation and Limitation Identification Logic	35
4.2	Result 1: wley $\rightarrow$ Domain Admin on INLANEFREIGHT	35
4.3	Result 2: Cross-Domain Trust Traversal	37
4.3.1	Cross-Forest Foreign MemberOf	38
4.4	Result 3: Constrained Delegation	39
4.5	Result 4: Documented Limitations	40
4.6	Discussion	40
4.6.1	Model Coverage and Training-Set Ceiling	40
4.6.2	Expressiveness vs. Transferability: HGT vs. GCN	41

4.6.3	Pipeline Completeness impact	41
5	Limitations and Future Work	43
5.1	Mathematical and Algorithmic Limitations	43
5.1.1	Multiplicative Probability Decay and Search Bias	43
5.1.2	Bounded Receptive Field and the Horizon Problem	44
5.1.3	The "Blind Compass" Effect in Lateral Movement	45
5.2	Operational and Data Boundaries	45
5.2.1	Edge Directionality and the Computer Node Sinkhole	45
5.2.2	Collector Incompleteness and Schema Evolution	46
5.2.3	ADCS Edge Sparsity and Static Stitching	46
5.3	Future Work: Algorithmic Upgrades	47
5.3.1	Waypoint Routing and Path Segmentation	47
5.3.2	Guided Search Heuristics (A^* Algorithm)	48
5.3.3	Terminal State Learning and Global Reachability	49
5.3.4	LLM Integration for Out-Of-Vocabulary (OOV) Edges	49
5.4	Conclusion	50
	General Conclusion	51
	Bibliography	52
	A Sanitised BloodHound JSON Example	54
	B Heterogeneous Graph Schema	55
	C Key Code Excerpts	56
	D GUI Screenshots	57

List of Figures

1.1	Active Directory object hierarchy.	4
1.2	Kerberos Authentication Flow	7
1.3	Active Directory Trust relationships.	8
2.1	Data Pipeline flow.	20
2.2	The logical architecture and domain topology of the GOADv2 environment. <i>Source: Mayfly (2022).</i>	23

List of Tables

2.1	Step cost assignment by attack category.	24
3.1	Core technical stack and version pinning for environment reproducibility.	31

List of Abbreviations

ADCS Active Directory Certificate Services. 2

DCSync Directory Replication Service synchronisation attack. 2

GOAD Game of Active Directory. 2

HGT Heterogeneous Graph Transformer. 2

PyG PyTorch Geometric. 2

RBCD Resource-Based Constrained Delegation. 2

SID Security Identifier. 2

General Introduction

Context

Active Directory is the identity backbone of major enterprises. While typically isolated withing the inside network, once the perimeter defenses are breached (e.g., via web service exploitation, phishing, or a compromised usb) an advarsary gains the ability to directly interacte and query the Active Directory Domain Controller. Through protocols such as LDAP and SMB the entirety of the organisation's IT infrastructure , data and services is essencially mapped out for the attacker. A vulnerable AD poses extreme levels of risk making hardening the AD network environment an extreme priority.

Once inside, attackers frequently leverage credential-based and ACL-based attacks such as pass-the-hash, DCSync, Kerberoasting are among the most impactful attack vectors in modern breaches. Red teams and security auditor alike rely on tools like BloodHound to map the attack surface and identify the vulnerable nodes during an engagement. However the analysis remains a highly manual process, time consuming, heavily dependant on human expertise and none scalable in large enterprise environments.

Problem Statement

Bloodhound maps the entirety of the Active Directory to a graph database, each node represents an object, and the relationships between these objects is represented by edges. When a user runs a path finding query, BloodHound simply runs Dijkstra over the attack graph, it finds the shortest path, not necessarily the most realistic or discreet one.

The question this thesis asks : can a model learn a navigation policy and generalise that policy to an unseen entreprise environment?

Objectives

The objectif of this Thesis is to provide a proof of concept for learned attack path discovery over heterogeneous Active Directory graphs.

To that end this work aims to:

- Build an end-to-end pipeline transforming raw BloodHound and Certipy JSON

to navigable [PyTorch Geometric \(PyG\)](#) [HeteroData](#) tensors.

- Train a GNN-based navigation policy and demonstrate cross-environment generalisation to unseen enterprise level environments.
- Identify and precisely describe the model’s limitations and architectural boundaries through systematic evaluation, establishing future work and improvements.

Main Contributions

- A complete, modular heterogeneous graph pipeline from raw BloodHound/Certipy JSON to navigable [PyG HeteroData](#) tensors, covering [Active Directory Certificate Services \(ADCS\)](#), [Resource-Based Constrained Delegation \(RBCD\)](#), [GPLink](#), and [Security Identifier \(SID\)](#) history edges.
- A [Heterogeneous Graph Transformer \(HGT\)](#)-based navigation policy trained on [Game of Active Directory \(GOAD\)](#) traces and evaluated on the entirely unseen INLANEFREIGHT enterprise environment, demonstrating generalisation from a 351-node lab to a 4 000-node network.
- An evaluation of the tool’s current limitations. Identifying how the search algorithm naturally penalizes long paths, which creates an unintended bias toward shorter, noisier attacks rather than longer, stealthier desired routes. A highlight of the model’s limited line of sight receptive field when trying to discover deep, multi-step continuous attack chains. Also acknowledging that testing was restricted to lab environments rather than enterprise networks due to time and resource constraints.
- A proposal for a future, second-generation tool that addresses these limitations by: implementing a guided search algorithm (such as A-Star) to help the model value stealth and keep long attack paths alive; using natural language AI to read and understand entirely new types of attacks based on their descriptions instead of relying on a hardcoded list; and adding a time dimension to the graph to accurately reflect temporary network states, such as active user sessions that log on and off. An automated "divide-and-conquer" routing system that breaks excessively long attack chains into smaller manageable segments using major network hubs or domain boundaries as checkpoints.

Report Outline

The remainder of this document is structured as follows.

Chapter 1 establishes the theoretical foundation, reviewing the architecture of Active Directory, common privilege escalation techniques, the machine learning logic behind GNNs, and the current state of graph-based attack path analysis.

Chapter 2 details the methodology and system architecture, introducing the heterogeneous graph pipeline designed to transform raw BloodHound data into navigable tensors. It also presents the core logic of the navigation policy, the loss function, and the beam search method.

Chapter 3 presents the internal architecture and the training paradigm of the Graph Neural Network GNN navigation policy.

Chapter 4 provides an empirical evaluation of the learned policy across multiple environments, documenting both successful generalizations and precise failure modes. It evaluates the tool’s current limitations, specifically identifying how the search algorithm naturally penalizes long paths (creating a bias toward shorter, noisier attacks) and defining the model’s limited receptive field when navigating deep attack chains.

Chapter 5 concludes the report by reviewing the achievements of this proof of concept and honestly assessing its limitations. It then proposes a second-generation tool featuring practical engineering improvements, such as automated waypoint routing to chunk long paths, guided search algorithms to prioritize stealth, and the integration of natural language models to dynamically interpret novel attack techniques.

Chapter 1

Background and Related Work

1.1 Active Directory: Architecture and Security Model

1.1.1 Core Objects and Structure

Active Directory (AD) is a directory service for Windows network environments. It is a distributed, hierarchical structure that allows for centralized management of an organization's resources, including users, computers, groups, network devices, file shares, group policies, servers and workstations, and trusts. AD provides authentication and authorization functions within a Windows domain environment.[1]

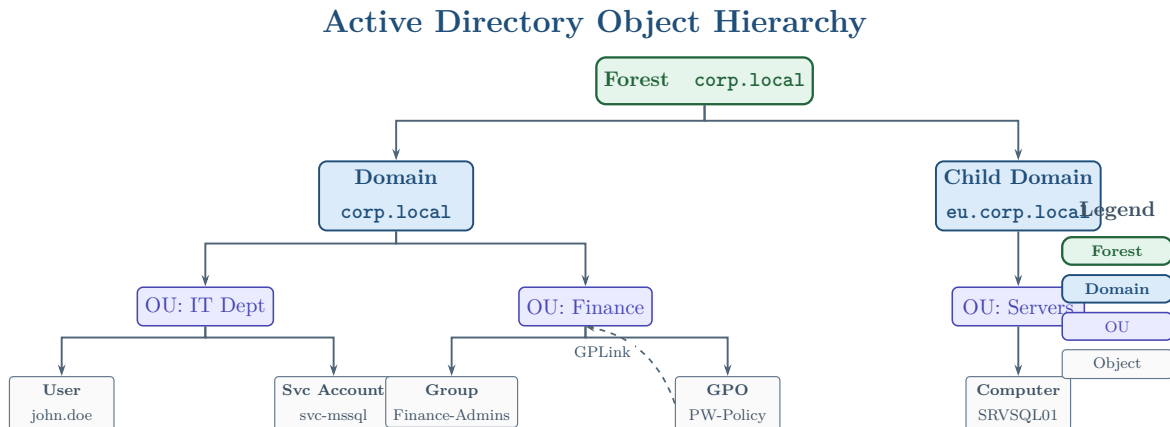


Figure 1.1: Active Directory object hierarchy.

Figure 1.1 illustrates the standard hierarchical structure of an Active Directory environment. At the highest level is the Forest, which serves as the absolute security boundary for the organization. Within the forest, one or more Domains act as logical partitions to manage resources. To streamline administration, these domains are further subdivided into Organizational Units OUs. OUs act as geographical or functional containers to group specific base objects, such as users, service accounts, security groups, and computers. Additionally, the diagram highlights how Group Policy Objects GPOs can be linked directly to OUs to enforce specific security configurations across all objects

within that container.

By design, Active Directory allows almost any authenticated user regardless of their administrative privilege level to read and query the directory. This open read-access is intended to facilitate normal network operations, such as allowing a user to look up a colleague's email address or locate a network printer. However, this feature creates a significant security vulnerability.

If an adversary compromises a single, unprivileged user account or workstation, they can abuse this read access using standard protocols like LDAP and RPC. The attacker can silently sweep the directory to extract the exact same list of computers, group memberships, trust relationships, and Access Control Lists (ACLs) detailed above. By stitching this data together, the adversary can map the entire network topology and identify hidden, multi-step privilege escalation paths leading from their starting point to high-value targets like Domain Admin.

Because the relationships between thousands of Active Directory objects create a highly complex web of permissions, manually identifying these chained vulnerabilities is nearly impossible. To address this, security professionals, defenders, and penetration testers utilize graph-based pathfinding tools. By ingesting the same directory data that an attacker would enumerate, these tools model the Active Directory environment as a mathematical graph. This allows operators to visualize hidden attack paths, proactively identify structural weaknesses, and remediate dangerous permission chains before they can be exploited during a real breach.

A standard Active Directory user account with no administrative privileges can be leveraged to query and enumerate a vast amount of structural and security data across the network. This accessible information includes, but is not limited to:

- Domain Computers
- Domain Group Information
- Default Domain Policy
- Password Policy
- Domain Trusts
- Domain Users
- Organizational Units (OUs)
- Functional Domain Levels
- Group Policy Objects (GPOs)
- Access Control Lists (ACLs)

1.1.2 Authentication Protocols

Active Directory manages network security through the fundamental AAA framework: Authentication, Authorization, and Accounting [2]. When a user, machine, or service

attempts to access a network resource, AD first verifies their cryptographic identity (Authentication). Once identity is proven, the system evaluates the user's embedded Security Identifier (SID) and group memberships against the target resource's Access Control Lists (Authorization). Finally, the Domain Controller logs the logon events and access requests (Accounting) to ensure auditability. To execute the initial authentication phase, Active Directory relies primarily on two distinct network protocols: Kerberos and NTLM.

Kerberos

Kerberos is the default and preferred authentication protocol in modern Active Directory environments [3]. It operates as a sophisticated, ticket-based system brokered by a Key Distribution Center (KDC), a service that runs natively on every Domain Controller. Rather than transmitting passwords or hashes across the network, Kerberos relies on symmetric-key cryptography. Upon a successful initial logon, the KDC issues the client a Ticket Granting Ticket (TGT). When the client needs to access a specific network service (such as a file share or database), it presents the TGT to the KDC to request a Service Ticket (ST) for that specific resource. While Kerberos relies on strict timestamping to prevent replay attacks and is cryptographically robust, its complex delegation and ticketing mechanics make it a primary target for advanced adversaries who abuse it via techniques such as Pass-the-Ticket, Golden Tickets, and delegation hijacking.

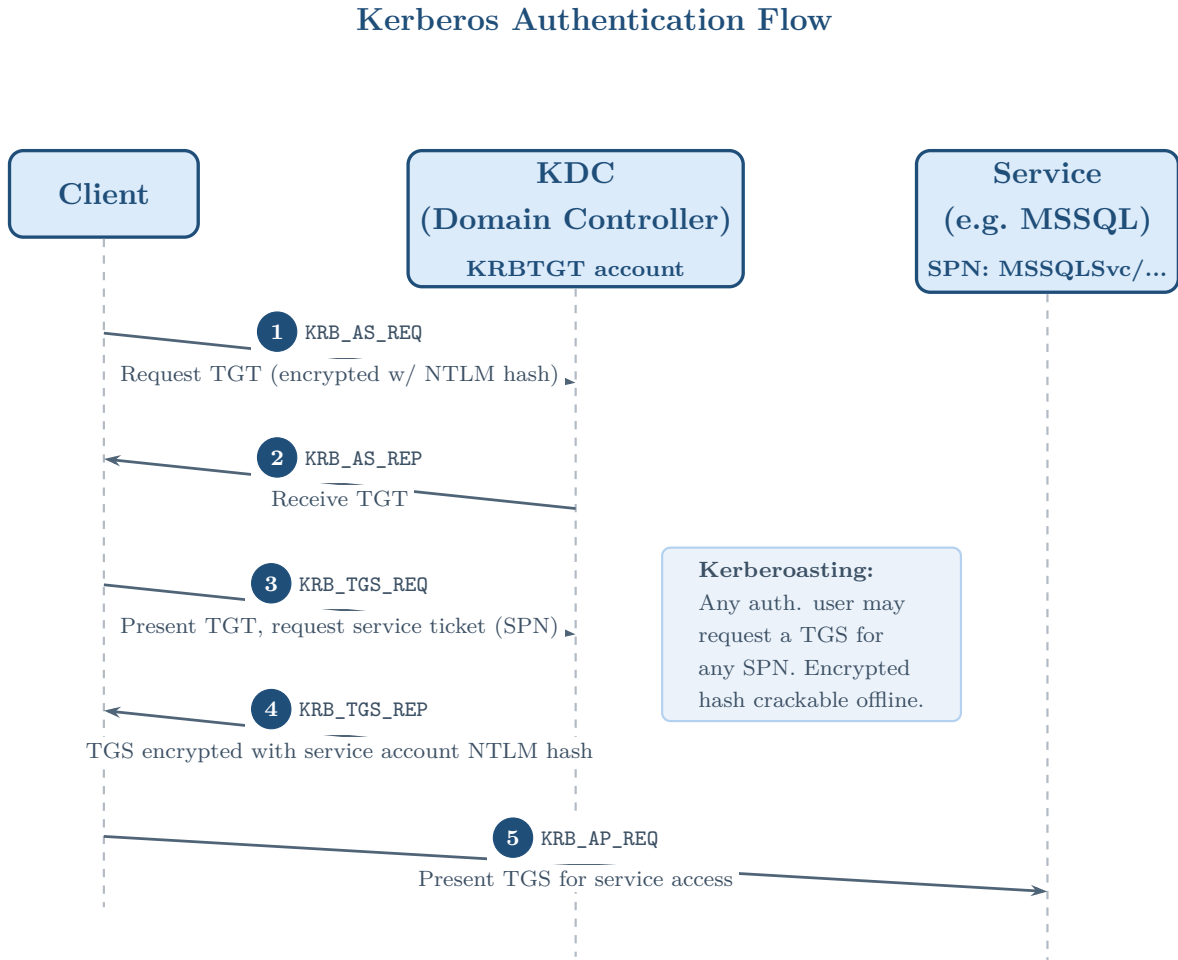


Figure 1.2: Kerberos Authentication Flow

Within the specific context of this thesis, two Kerberos-related dynamics are of critical offensive relevance. First, Constrained Delegation allows a service to request a Service Ticket on behalf of any user. This abuse primitive is captured by our data extraction pipeline as an `ALLOWEDTODELEGATE` edge, enabling the highly stealthy privilege escalation chains that our model learns to navigate. Second, while Kerberos secures standard authentication, ultimate domain compromise often bypasses ticket exchanges entirely. For instance, the `DCSync` attack leverages `GETCHANGES` and `GETCHANGESALL` replication rights to extract NTLM hashes directly from the Domain Controller without ever interacting with the Kerberos KDC.

NT LAN Manager (NTLM)

NT LAN Manager (NTLM) is a legacy suite of Microsoft security protocols that remains widely enabled in Active Directory environments, primarily to ensure backwards compatibility with older clients or poorly configured applications. Unlike the ticket-brokered architecture of Kerberos, NTLM utilizes a direct challenge-response mechanism. When a client attempts to authenticate to a server, the server issues a random challenge.

The client then encrypts this challenge using the NTLM hash of the user’s password and returns it to prove their identity. Because NTLM lacks mutual authentication and transmits cryptographic material in a predictable challenge-response format, it is inherently weaker than Kerberos. Offensively, NTLM is frequently targeted for exploitation through hash extraction (Pass-the-Hash) and network interception (NTLM Relay attacks).

1.1.3 Trust Relationships

In large enterprise environments, organizations frequently span multiple domains or even entirely separate Active Directory forests. To allow users in one domain to access resources located in another, Active Directory employs a mechanism known as Trust Relationships [2]. A trust acts as a secure cryptographic bridge between two boundaries. These relationships can be configured as one-way, where Domain A trusts Domain B (but not vice versa), or two-way, where both domains trust each other equally. Furthermore, trusts can be transitive—meaning the trust flows through a chain of domains—or non-transitive, restricting the relationship strictly to the two interacting domains. As illustrated in the diagram below, these trust links can occur between child and parent domains within the same forest, or across entirely different corporate forests.

Forest and Trust Relationships

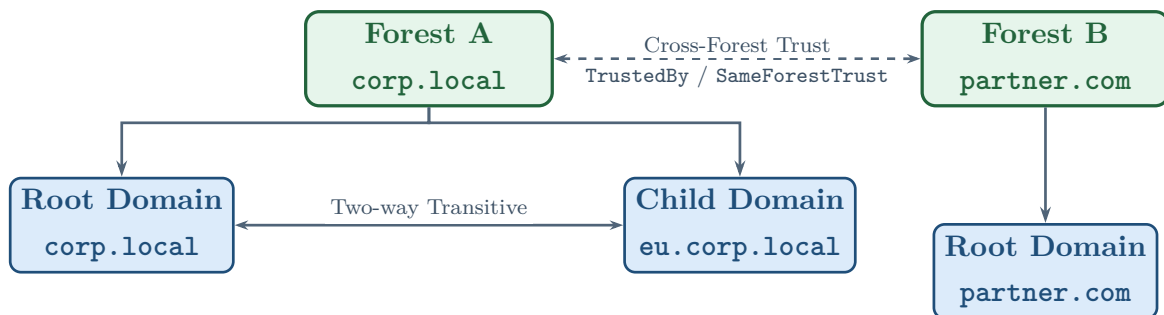


Figure 1.3: Active Directory Trust relationships.

While trust relationships are essential for administrative flexibility and resource sharing, they introduce complex attack vectors when misconfigured or overly permissive [4]. Adversaries frequently abuse bidirectional transitive trusts to move laterally across an entire forest boundary. For instance, if an attacker compromises a loosely secured child domain, they can forge an inter-realm ticket or abuse the SID History attribute to escalate privileges to the parent root domain, effectively compromising the entire forest. Similarly, poorly audited cross-forest trusts can inadvertently grant external users excessive permissions. This allows an attacker who compromises a partner organization’s network to pivot seamlessly into the primary corporate environment.

Consequently, mapping and analyzing these trust links is a critical component of assessing an organization’s true security perimeter.

1.2 Privilege Escalation in AD Environments

1.2.1 Offensive ACL Abuse

Access Control Lists (ACLs) serve as the granular security foundation of Active Directory, defining exactly which principals have specific permissions over directory objects. Historically, defenders focused primarily on network vulnerabilities or missing software patches. However, modern offensive tradecraft has demonstrated that abusing misconfigured ACLs is often the most common, reliable, and critical path for privilege escalation within an enterprise environment [5].

Because ACL misconfigurations allow attackers to subvert the intended security architecture seamlessly without triggering traditional malware alerts, mapping these specific permissions is the primary function of data collection tools like BloodHound. Consequently, these structural ACL relationships form the foundational dataset that the GNN-AD-Navigator pipeline extracts. Our model heavily bases its recommended paths on these extracted rights, learning to weigh the operational cost of exploiting each specific permission.

When an adversary compromises a basic account, they enumerate the directory to identify objects where their controlled principal holds abusive rights. The most critical offensive ACL primitives include:

- **GenericAll:** This right grants complete, unrestricted control over the target object. An attacker holding **GenericAll** can execute any action, such as adding a user to a highly privileged group, resetting an account’s password, or modifying critical attributes to establish persistence.
- **WriteDACL:** This permission allows a principal to modify the target object’s access control list itself. From an offensive perspective, an attacker can abuse **WriteDACL** to simply grant themselves **GenericAll** privileges, effectively taking full control of the target in a two-step process.
- **GenericWrite:** While not providing full control, this right allows an attacker to update specific properties of an object. It is frequently weaponized to modify the `servicePrincipalName` (SPN) attribute to enable targeted Kerberoasting, or to alter the `msDS-AllowedToActOnBehalfOfOtherIdentity` attribute to execute Resource-Based Constrained Delegation (RBCD) attacks.
- **ForceChangePassword:** This specific permission allows an attacker to reset the password of a target user without knowing the current password. While highly

effective for immediately compromising an account and hijacking its session, it is operationally “loud.” It disrupts the legitimate user’s access, making it highly detectable by defenders.

- **AddMember:** If held over a security group, this right allows an attacker to directly append their own controlled account (or another compromised user) into the target group. This immediately grants the attacker all the privileges inherited by that group across the entire domain.

1.2.2 DCSync Attack

The DCSync attack is one of the most critical techniques used by adversaries to extract credentials from a domain without requiring direct code execution on a Domain Controller [6]. Active Directory relies on the Directory Replication Service Remote Protocol (MS-DRSR) to synchronize data across multiple Domain Controllers. To abuse this, an attacker only needs an account that holds specific replication permissions—specifically, `GETCHANGES` (Replicating Directory Changes) and `GETCHANGESALL` (Replicating Directory Changes All) at the domain root level.

Once these permissions are acquired, the attacker’s machine can masquerade as a legitimate Domain Controller and issue a replication request. The target DC will respond by transmitting the requested account credentials, including the highly sensitive NTLM hash of the `krbtgt` account. Securing these replication rights is often the terminal objective in offensive pathfinding operations, as it allows for the immediate creation of Golden Tickets and total forest compromise.

1.2.3 ADCS Attack Paths

Active Directory Certificate Services (ADCS) is Microsoft’s native Public Key Infrastructure (PKI) implementation. While it provides essential cryptographic functions, misconfigured certificate templates introduce devastating escalation vectors [7]. These vulnerabilities, famously categorized into escalating scenarios (ESC1 through ESC15), often allow an unprivileged user to request a certificate on behalf of a highly privileged principal, such as a Domain Admin.

By leveraging the PKINIT protocol, the attacker can use the fraudulently obtained certificate to request a Kerberos Ticket Granting Ticket (TGT) for the impersonated admin, instantly seizing control of the domain. The inclusion of ADCS nodes and edges (such as `ENROLL` and `MANAGECA`) drastically expands the complexity of the Active Directory attack graph, presenting massive new navigational challenges for automated pathfinding tools.

1.2.4 SID History Abuse and RBCD

Beyond standard access controls, advanced adversaries frequently abuse inherent Active Directory features designed for compatibility and migration. The `sIDHistory` attribute, originally intended to preserve a user's access rights when migrating between domains, can be weaponized. If an attacker compromises a domain, they can inject the Security Identifier (SID) of a highly privileged group (such as Enterprise Admins) into an account's `sIDHistory`. This grants the attacker persistent, cross-domain administrative access while bypassing standard group membership audits.

Similarly, Resource-Based Constrained Delegation (RBCD) shifts the control of delegation from the domain administrator to the target resource itself [8]. If an attacker acquires `GENERICWRITE` privileges over a target computer object, they can modify its `msDS-AllowedToActOnBehalfOfOtherIdentity` attribute. This modification allows the attacker to authorize a computer they already control to impersonate any user on the compromised target, effectively granting them full administrative access to the machine. Because RBCD relies on targeted attribute modification rather than global delegation rights, it represents a stealthy and highly effective edge in modern attack graphs.

1.3 Existing Tools and Their Limitations

1.3.1 BloodHound and SharpHound

BloodHound has become the de facto industry standard for visualizing and analyzing Active Directory security structures [9]. Operating on a Neo4j graph database backend, BloodHound ingests data collected by its C# ingestor, SharpHound, to map relationships between users, computers, and groups. By representing AD components as nodes and privileges as edges, BloodHound allows security professionals to visualize complex attack paths that would be impossible to identify manually.

However, BloodHound's primary limitation lies in its routing logic. The platform relies heavily on Dijkstra's algorithm to calculate the shortest path between an attacker's starting node and their target. In this deterministic model, every edge is treated with an equal, unweighted cost of one. BloodHound cannot natively distinguish between a highly detectable, noisy action (such as forcing a password reset on an active administrator) and a highly stealthy vector (such as abusing an `ALLOWEDTODELEGATE` trust). Because it strictly minimizes hop count, the tool frequently recommends operationally unviable or easily detectable attack paths, leaving a critical gap for context-aware, machine learning-driven routing solutions.

1.3.2 Certipy

SharpHound historically lacked deep inspection capabilities for certificate infrastructure. To address this, **Certipy** was developed as a Python-based equivalent specifically for **ADCS**, designed for cross-platform and remote execution. Created by Oliver Lyak, Certipy is explicitly defined as an “offensive tool for enumerating and abusing Active Directory Certificate Services” [10]. Developed primarily for UNIX-like environments, this tool leverages remote queries to extract certificate templates, Certificate Authorities, and complex PKI permissions. It is an essential component for attackers operating from external infrastructure or non-Windows pivot machines, and it serves as the primary **ADCS** data collection engine for the pipeline developed in this thesis.

1.3.3 bloodhound-python

While SharpHound is designed to be executed directly from a compromised Windows host, **bloodhound-python** serves as its Python-based equivalent, designed for cross-platform and remote execution [11]. Developed primarily for UNIX-like environments, this tool leverages the **impacket** library to query LDAP and RPC remotely. It is an essential component for attackers operating from external infrastructure or non-Windows pivot machines, and it serves as the primary data collection engine for the pipeline developed in this thesis.

However, **bloodhound-python** suffers from severe functional limitations compared to its C# counterpart. Most critically, it lacks native support for enumerating **ADCS** templates. To bridge this gap, operators are forced to rely on external tooling such as **Certipy**. This introduces a fragile dependency chain: because Certipy officially retired BloodHound integration in its v5.0.0 release, operators must intentionally downgrade and maintain legacy versions (e.g., v4.3.0) strictly for dataset compatibility [12].

Furthermore, **bloodhound-python** exhibits blind spots in complex enterprise architectures, notably struggling to accurately map specific cross-domain trust accounts. Because its default implementation relies on static and often lagging extraction logic, the ingester will blindly bypass novel misconfigurations or unmapped edge types unless its underlying parsing code is manually updated. This effectively blinds any downstream pathfinding algorithms to those specific routes, cementing the pipeline’s extraction phase as a critical bottleneck for the overall model.

1.3.4 Other Automated Attack Path Tools

The success of BloodHound has inspired several supplementary tools aimed at automating attack path execution and reporting. Tools such as PlumHound and GoFetch interact directly with BloodHound’s Neo4j database to generate human-readable audit reports or to automatically execute the identified exploit chains [13]. Similarly, auditing

software like PingCastle provides rapid, rule-based assessments of Active Directory risk scores by flagging known misconfigurations [14].

While these tools excel at operationalizing graph data and generating compliance reports, they fundamentally inherit the baseline limitations of the underlying Neo4j Cypher queries. They do not generate novel paths or apply dynamic, environment-specific heuristics to evaluate edge costs. The offensive security industry currently lacks a widely adopted tool capable of applying heterogeneous attention mechanisms to weigh the semantic value of AD relationships. This thesis addresses this exact void by substituting deterministic shortest-path algorithms with a **Graph Neural Network (GNN)** policy capable of learning stealth and operational viability.

1.4 Graph Neural Networks

1.4.1 Message Passing and Neighbourhood Aggregation

Standard deep learning models are generally designed for structured data, such as grids of pixels or sequences of text. However, Active Directory (AD) environments are highly interconnected and asymmetrical, making them best represented as graphs. Graph Neural Networks (GNNs) address this by learning directly from the structural relationships between entities in the network.

Let an Active Directory environment be defined as a graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, where $\mathcal{V}$ represents the set of nodes (users, groups, computers) and $\mathcal{E}$ represents the set of edges (permissions, trusts, active sessions). The primary objective of a GNN is to encode each node into a continuous numerical vector, a **node embedding**. The initial state of a node $v \in \mathcal{V}$ is denoted by its starting feature vector, $\mathbf{h}_v^{(0)} = \mathbf{x}_v$.

To capture the topology of the network, GNNs rely on a mechanism called *message passing*. During this process, each node gathers information from its direct neighbours to update its own state. Mathematically, the operation at the k -th **hidden layer** consists of an aggregation step and an update step. The message collected from the neighbourhood $\mathcal{N}(v)$ is defined as:

$$\mathbf{m}_{\mathcal{N}(v)}^{(k)} = \text{AGGREGATE}^{(k)} \left(\left\{ \mathbf{h}_u^{(k-1)} : u \in \mathcal{N}(v) \right\} \right)$$

where AGGREGATE is a mathematical function such as the sum or mean. After aggregating this neighbourhood data, the node updates its own embedding. It combines its previous state with the newly collected message using a learned weight matrix $\mathbf{W}^{(k)}$ and a non-linear activation function σ :

$$\mathbf{h}_v^{(k)} = \sigma \left(\mathbf{W}^{(k)} \cdot \left(\mathbf{h}_v^{(k-1)} \parallel \mathbf{m}_{\mathcal{N}(v)}^{(k)} \right) \right)$$

The number of hidden layers, K , determines the receptive field of the model—meaning

how far a node can "see" into the graph. For example, in a network with $K = 3$ layers, the final embedding $\mathbf{h}_v^{(K)}$ captures structural information from nodes up to three hops away.

Once the node embeddings are finalized, they are used to make routing decisions during an attack path simulation. This is managed by the **policy head**. Since the number of outbound connections varies greatly across AD nodes, the policy head evaluates each potential neighbour individually. It concatenates the current node's embedding $\mathbf{h}_v^{(K)}$, the candidate neighbour's embedding $\mathbf{h}_u^{(K)}$, and the target node's embedding $\mathbf{h}_{\text{target}}^{(K)}$, passing them through a Multi-Layer Perceptron (MLP) to generate a raw transition score:

$$e_{v,u} = \text{MLP} \left(\mathbf{h}_v^{(K)} \parallel \mathbf{h}_u^{(K)} \parallel \mathbf{h}_{\text{target}}^{(K)} \right)$$

These raw scores are then normalized across all valid neighbours using a Softmax function. This produces a valid probability distribution π that guides the pathfinding algorithm toward the most promising next step:

$$\pi(u|v) = \frac{\exp(e_{v,u})}{\sum_{w \in \mathcal{N}(v)} \exp(e_{v,w})}$$

The Impact of Graph Size and Density

While GNNs are highly effective, their performance is closely tied to the graph's size and connectivity. In large Active Directory networks, dense configurations—such as heavily nested group memberships—can cause *neighbourhood explosion*. This drastically increases the computational overhead during the message passing phase.

Additionally, increasing the number of hidden layers to expand the receptive field can trigger *over-smoothing*. In this scenario, repeated aggregation causes the embeddings of structurally distinct nodes (e.g., a standard workstation and a Domain Controller) to blend together and lose their unique characteristics. Therefore, balancing the number of layers is crucial to maintaining the model's accuracy.

Finally, standard GNN architectures assume that all nodes and edges share the same properties. To properly model the diverse object types and distinct permission structures found in Active Directory, this foundation must be extended into a Heterogeneous Graph Transformer (HGT).

1.4.2 Heterogeneous Graph Transformer (HGT)

While a standard [Graph Convolutional Network \(GCN\)](#) aggregates neighbourhood features uniformly, Active Directory graphs are fundamentally heterogeneous. A MEMBEROF relationship carries drastically different operational semantics and stealth im-

plications than an `ALLOWEDToDELEGATE` or `WRITEDACL` edge. The critical need for context-aware routing is currently driving rapid innovation in the offensive security space; for instance, recent 2025 research applying Physics-Informed GNNs to Active Directory graphs achieved an F1 score of 0.9308 when predicting paths across 16 distinct offensive edge types [15].

To capture this required semantic diversity, this research employs the `HGT` architecture proposed by Hu et al. [16]. `HGT` introduces a type-specific attention mechanism. Instead of sharing weights globally, it computes attention based on the specific meta-relation triplet $\tau = (\tau(s), \tau(e), \tau(t))$, representing the source node type, edge type, and target node type. The mutual attention score between a source node s and a target node t is calculated by projecting their embeddings into distinct Query and Key spaces via type-specific linear projections:

$$\text{Attention}(s, t) = \text{Softmax}_{\forall s \in \mathcal{N}(t)} \left(\parallel_{i \in [1, h]} \frac{\mathbf{K}_s^{(i)} \mathbf{W}_{\tau(e)}^{(i)} \mathbf{Q}_t^{(i)T}}{\sqrt{d/h}} \right)$$

This formulation allows the network to dynamically learn which specific attack vectors are most relevant when navigating between different entity types. Regarding the specific architecture implemented in this pipeline, the model utilizes an embedding dimension of 64, 4 attention heads, and is constrained to only 2 hidden layers. These hyperparameters act as a deliberate regularization strategy. The primary deciding factors were the physical size of the `GOAD` dataset during training, and the strict need to prevent the severe over-smoothing that occurs when aggregating features across dense Active Directory topologies.

1.4.3 PyTorch Geometric and HeteroData

To implement these mathematical frameworks, this project leverages PyTorch Geometric (PyG), a specialized extension library for deep learning on irregularly structured data **fey2019pyg**. Within PyG, the AD graph is constructed using the `HeteroData` format.

Traditional graph libraries store all connections in a single adjacency matrix, fundamentally stripping away semantic meaning. In contrast, the `HeteroData` architecture partitions the graph into distinct dictionaries based on meta-relations. Each relationship type—such as `(user, memberof, group)`—maintains its own isolated `edge_index` tensor of shape $[2, |\mathcal{E}_\tau|]$.

This isolated tensor structure is uniquely suited to the modern trajectory of offensive security. The industry is rapidly moving beyond isolated Windows domains; notably, the 2025 release of BloodHound Community Edition v8 introduced “OpenGraph,” which expands pathfinding to include hybrid cloud identities, Entra ID, and third-party platforms like GitHub [17]. A homogeneous adjacency matrix would collapse under

the introduction of these vastly different node and edge types. **HeteroData** natively supports this expansion by simply allowing the injection of new relationship dictionaries without restructuring the existing graph.

Furthermore, decoupling the graph structure from the ingestion logic provides massive pipeline resilience. As legacy data collection scripts face deprecation—such as Kali Linux 2025 enforcing strict Python 3.13 environments that break older Neo4j drivers [18]—our PyG-based pipeline remains agnostic. By storing the graph purely as isolated PyTorch tensors rather than relying on an active database connection, the GNN can cleanly mask out irrelevant relationships and accommodate future data collection tools with minimal friction.

1.5 Summary and Positioning

The application of deep learning to cybersecurity has historically been constrained by the rigid, Euclidean data structures required by traditional neural networks. However, enterprise networks are inherently non-Euclidean; they are highly complex webs of varying entities (users, computers, groups, policies) and heterogeneous relationships (authentication sessions, explicit permissions, cryptographic trusts). Graph Neural Networks represent the perfect theoretical fit for this domain, as they are natively capable of ingesting and learning from this structural and semantic diversity.

Despite this architectural alignment, the majority of applied GNN research in cybersecurity has been heavily skewed toward defensive anomaly detection. Models such as HetGLM [19] have demonstrated the efficacy of utilizing heterogeneous graphs to spot irregular lateral movement and anomalous authentication links. While highly valuable for Security Operations Centers (SOCs), these approaches model attacker behavior retrospectively. They attempt to catch active intrusions probabilistically rather than mapping structural vulnerabilities proactively.

More recently, literature has begun to explore GNNs for proactive attack path prediction, notably through Physics-Informed Graph Neural Networks [15]. However, these approaches frequently suffer from a purely mathematical, theory-driven bias. Because these models are often developed outside the context of applied offensive security, they rely heavily on auto-generated, synthetic datasets that fail to replicate the messy idiosyncrasies of real-world Active Directory deployments. Furthermore, rather than learning the true operational cost of a specific exploit—such as weighing the stealth of an `ALLOWEDTODELEGATE` abuse against the high detectability of a `FORCECHANGEPASSWORD` action—these models often devolve into simply replicating BloodHound’s shortest-path Dijkstra logic across larger matrices.

This thesis directly addresses that gap. Rather than approaching the directory graph as a pure mathematical traversal problem on synthetic data, this research positions

the GNN as an offensive agent. By leveraging the Heterogeneous Graph Transformer architecture and training it against operationally accurate environments, the pipeline developed in this work learns a nuanced offensive navigation policy. It does not simply seek the mathematically shortest path; it learns the most operationally viable route, effectively bridging the gap between academic graph theory and practical penetration testing tradecraft.

Chapter 2

System Architecture and Design

2.1 System Overview

The GNN-AD-Navigator is designed to process and analyze any [Active Directory \(AD\)](#) environment. It maps raw BloodHound scans into traversable [PyG](#) nodes and edges, allowing the model to navigate and score potential attack transitions. Furthermore, given a BloodHound-compatible Certipy scan, the tool is capable of stitching missing [ADCS](#) templates back into a unified [PyG](#) graph.

To achieve this, the pipeline utilizes a series of targeted scripts to construct the final `heterodata.pt` tensor object. These scripts handle the merging of dissected scans (for example, scans representing different domains or collected from different privilege levels), the injection of [ADCS](#) templates, and the cleanup of dangling nodes and edges. A validation script then ensures data integrity while generating clear, readable logs.

Operational Modes and Design Decisions

To accommodate different stages of a red team engagement, the pipeline was built with three primary operational modes, managed via the `launch.sh` wrapper:

1. **Data Preparation Only:** Parses the raw JSON files, stitches the graphs, and builds the tensors without querying the AI.
2. **Preparation and Query:** Builds the tensors and immediately launches the inference script from a specified start node to a target node.
3. **Inference Only:** Once the `heterodata.pt` object is created the user can skip the data preparation phase and run the pathfinding algorithm directly.

A principal design decision maintained throughout this pipeline is the enforcement of the BloodHound-standard principal-to-target edge directionality. This ensures that the simulated attacker can only traverse edges where they mathematically hold the source privilege.

Path Discovery and Auditing

Once the graph accurately mirrors the target [AD](#) environment, the inference phase begins. The `beam_search` algorithm queries the model’s policy head to navigate the graph, discovering viable attack paths and ranking them based on the network’s confidence scores. Finally, the tool runs an operational audit against static rules, evaluating the output and reminding the user to carefully review each path before proceeding with active exploitation.

2.2 Data Collection and Preprocessing

The `launch.sh` script serves as the primary execution pipeline for the GNN-AD-Navigator, orchestrating both the data preparation and model inference phases. During the preparation stage, it categorizes raw JSON outputs from BloodHound and Certipy, cleans the data, and merges it into a unified Active Directory forest graph. It then translates this combined graph into tensor formats `heterodata.pt` required by PyTorch. Finally, the script can execute an inference query using either a GCN or HGT model to calculate potential attack paths between a user-defined starting node and target node.

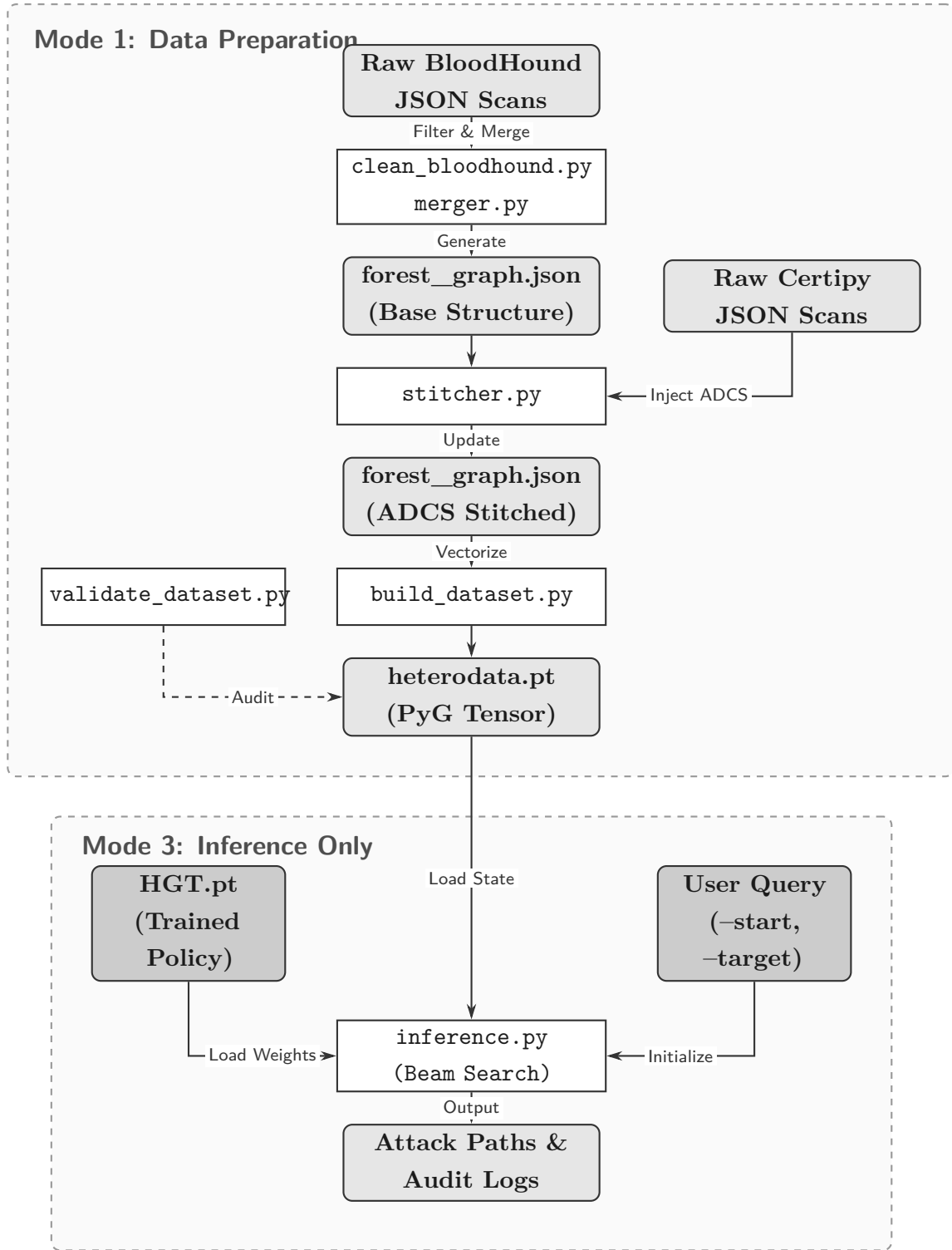


Figure 2.1: Data Pipeline flow.

2.2.1 clean_bloodhound.py: Filtering and Multi-Domain Accumulation

The initial stage of the data preparation pipeline is handled by the `clean_bloodhound.py` script, which is responsible for sanitizing raw BloodHound JSON outputs. To support large-scale enterprise environments, the script implements a recursive, multi-domain ac-

cumulation mechanism. This ensures that when input data contains scans from multiple domains or is split across various files, no topological data is silently overwritten. The script accomplishes this by deduplicating entities based on their `ObjectIdentifier` (`SID`), selectively retaining the “richer” JSON record if duplicate scans of the same domain occur.

Crucially, this component acts as a noise-reduction filter to optimize the `GNN`’s computational overhead. It builds an initial edge index across all entities to determine topological connectedness. Using this index, the script systematically drops irrelevant nodes. For example, `Computer` nodes matching standard workstation naming conventions (e.g., `WKS-0`) are pruned if they lack dangerous `Access Control List (ACL)` privileges, unconstrained delegation rights, and possess an edge degree of one or less. Similarly, disabled `User` nodes are purged from the dataset unless they retain active sessions or explicit `ACL` edges, while empty `Group` objects with no members or outbound privileges are entirely discarded. Nodes representing Domains, `Group Policy Objects (GPOs)`, and `Organisational Units (OUs)` are preserved as-is to maintain structural hierarchy.

2.2.2 `merger.py`: Normalisation and Merging

Following the initial filtering and `ADCS` data stitching, the pipeline executes `merger.py`. This script functions as a generic forest merger, capable of accepting heterogeneous inputs such as directories of cleaned JSON files or previously stitched domain graphs and consolidating them into a unified `forest_graph.json` structure. Positioned directly before the tensor construction phase, this module is strictly responsible for cryptographic and structural alignment.

To ensure flawless tensor mapping in PyTorch Geometric, the script enforces a strict normalization protocol. All `ObjectIdentifier` and `PrincipalSID` values, including cross-domain `TargetDomainSid` references, are strictly cast to uppercase, while key categorical properties (such as `name`, `domain`, and `RightName`) are normalized to lowercase.

When merging overlapping datasets, the script resolves conflicts through a deterministic scoring heuristic. The `score_obj` function evaluates node richness by awarding points for populated properties, active `ACL` rules, and group members, while applying a heavy multiplier to explicit cross-domain `Trusts`. The conflicting entity with the highest topological score is retained. Finally, the script executes a validation routine that inventories all discovered trust relationships, aggregates edge type frequencies, and calculates a “dangling-reference rate” to quantify relational integrity before the data is vectorized into the `GNN`.

2.2.3 `stitcher.py`: ADCS Injection and Domain Detection

Because Certipy operates independently of BloodHound, its raw JSON output lacks the native topological bindings required to merge seamlessly into an existing [AD](#) graph. To bridge this gap, the pipeline employs `stitcher.py`, an intermediate augmentation script positioned directly between the filtering and merging phases. Before the script is even invoked, the pipeline’s overarching wrapper automatically parses the Certipy output to dynamically detect the target domain. By analyzing the Certificate Subject or Distinguished Name embedded within the scanned certificates, the wrapper extracts the domain context and passes it to the stitcher without requiring manual operator input.

Once invoked, the script injects two novel node categories into the graph: [Certificate Authority \(CA\)](#) and `certtemplates`. Because Certipy does not natively retrieve [SIDs](#) for these entities, the script generates deterministic, synthetic Ghost [SIDs](#) (e.g., `S-1-5-21-GHOST-...`) by computing an MD5 hash of the template name and the target domain. To connect these isolated nodes to the broader network, the script builds a resolution dictionary that maps lowercase principal names to their existing `ObjectIdentifier` within the forest. It then translates complex [ADCS](#) relationships into standard BloodHound [Aces](#) entries, injecting edges such as `ENROLL`, `PUBLISHEDTO`, and `WRITEDACL`. Critically, it also injects an `HTTPENROLL` edge to model ESC8 vulnerabilities, and a `TRUSTEDFORNTAUTH` edge to link the [CA](#) back to the domain root, preventing the routing algorithm from dead-ending.

Beyond explicit [ADCS](#) integration, `stitcher.py` acts as the engine for synthetic [Advanced Persistent Threat \(APT\)](#) vector generation. It proactively scans the forest for logically vulnerable delegation configurations. If a source entity possesses `unconstraineddelegation` rights, the script dynamically generates a `COERCETGT` edge to mathematically represent [NT LAN Manager \(NTLM\)](#) coercion attacks (e.g., [Petit-Potam](#)). To preserve topological integrity, this script strictly validates that the target is an unprotected Domain Controller computer account, strictly preventing the [GNN](#) from calculating illogical attack paths through abstract entities like [OUs](#) or standard user accounts.

2.2.4 Training Environment: GOAD

The Game of Active Directory (GOAD), developed by Mayfly, serves as the primary training and diagnostic environment for this research. GOAD is a highly complex, multi-domain, and multi-forest vulnerable lab designed specifically to simulate advanced, real-world enterprise architectures. The environment is composed of five virtualized Windows Server machines running Windows Server 2016 and 2019 distributed across two distinct forests. The primary forest contains a parent domain `sevenkingdoms.local` and a child

domain `north.sevenkingdoms.local`, simulating a typical corporate hierarchy with bidirectional transitive trusts. A secondary, external forest `essos.local` is connected via a forest-to-forest trust with SID history enabled. This sophisticated topological layout provides the heterogeneous graph structure required to train the Graph Neural Network on cross-domain lateral movement and complex trust exploitations.

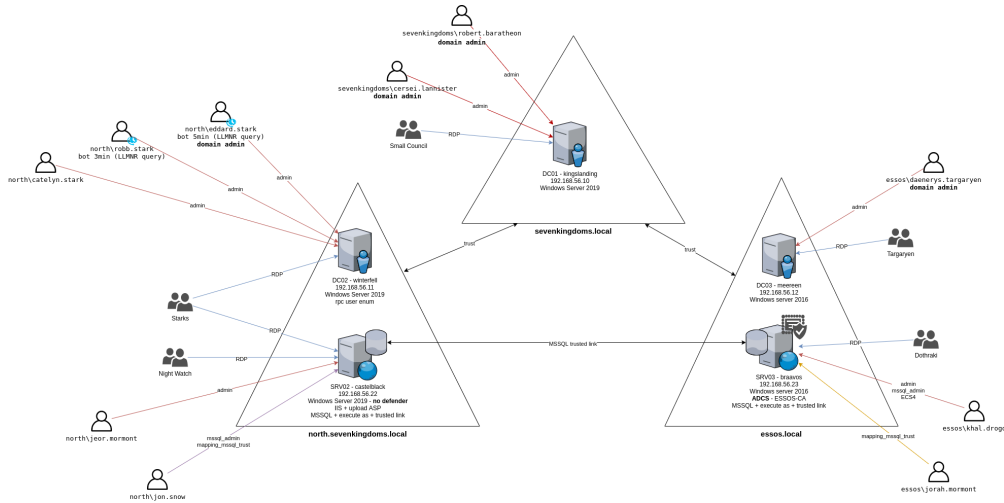


Figure 2.2: The logical architecture and domain topology of the GOADv2 environment. *Source: Mayfly (2022).*

Beyond basic network topography, GOAD is engineered with deeply embedded, configuration-level vulnerabilities that test the limits of automated attack path discovery. The environment features multi-step [ACL](#) kill-chains, deliberately exposed [ADCS](#) configured to be vulnerable to ESC1 through ESC3 and ESC8 relay attacks, and various forms of delegation (Unconstrained, Constrained, and Resource-Based Constrained Delegation). By exposing the Navigation Model to this dense concentration of intersecting vulnerabilities, the network learns to evaluate competing escalation routes—such as choosing between a force change password against an IIS server or a stealthy `WriteDACL` abuse—making GOAD an ideal foundational dataset for imitation learning in offensive security. The attack paths goad provides are to be presented in a learnable and readable training format for the model. Each attack path is deconstructed into transitions, `current_node`, `next_hop`, `credentials_used`, `total`, `path_weight`, `train_on_step`, `notes` and the `step_cost`.

2.2.5 Feature engineering

To a GNN model learning from a bloodhound scan, attack paths are of different importance, some are totally invisible while others are a gold mine for what the model needs to replicate. Paths that rely on unpatched vulnerabilities or outdated systems, for instance, leave no trace in the BloodHound graph and are marked with a `path_weight` of 0. Paths that are mostly visible with only two or less transitions that do not

qualify have the latter marked through `train_on_step` set to zero. And at last, for an imitation learning model, given the choice between two valide transitions, the model choses based on training data, prefering the edge type it has seen most during training. To address this We are to more or less guide the model to gravitate towards the edge attack with less impact on the environment, the one with less noise and impact. To reach the desired effect each valid transition is scored. This score is reflected directly in the loss function through the `step_cost` feature.

Table 2.1: Step cost assignment by attack category.

Attack Category	Cost Range	Rationale
Topological / Logical	0.1 – 0.2	Operations such as adding a member to a group or enrolling in a certificate template. These transitions are structurally clean and rarely trigger alerts unless heavy auditing is configured.
Credential Abuse	0.3 – 0.4	Using a recovered password or a forged TGT. Low noise overall, but detectable by honeytokens or behavioural analysis tools.
Active Enumeration	0.5	Kerberoasting or intensive LDAP queries. Intermediate noise level; commonly detected by modern SIEMs monitoring ticket requests or query volume anomalies.
Memory / Disk Dumping	0.6 – 0.7	Dumping the SAM database or LSASS process memory. High detection risk; endpoint protection tools such as Microsoft Defender and CrowdStrike flag these operations near-immediately.
Network Exploitation	0.8 – 0.9	Active coercion and relay attacks such as PetitPotam or NTLM Relaying, and brute-force techniques such as password spraying. Very noisy; triggers IDS/IPS rules and account lockout policies.

The `step_cost` value assigned to each transition feeds directly into the loss function through the effective weight computation:

The feature engineering phase introduced three mechanisms: filters to eliminate CVE-dependent transitions invisible to the graph collector; path weights reflecting a composite visibility score that prioritises transitions where the graph structure itself is the primary escalation vector; and a `step_cost` feature guiding the model toward stealthier, less impactful edges when a choice presents itself. This constitutes a simplified reward shaping approach, consistent with standard practices in the imitation learning literature.

2.3 Navigation Model

2.3.1 HGT Encoder

To effectively process the highly asymmetrical and heterogeneous structure of `AD`—which contains distinct entity types (e.g., users, computers, groups, domains) alongside a complex vocabulary of privilege edges—the architecture utilises an `HGT` encoder [20]. Implemented via the `PyG` library, the encoder is specifically designed to handle the native topological variations without requiring manual modification of the graph, preserving the distinct semantic meaning of each node and edge type as established in the foundational `HGT` literature.

The encoding phase begins by projecting the disparate input features of each node type into a unified dimensional space. This is achieved by passing the raw features through type-specific linear projection networks, mapping every object into a standardised 64-dimensional hidden vector. Once projected, the architecture applies a series of `HGT` convolutional layers utilising multi-head attention mechanisms to propagate context across the network. Specifically, the model employs four attention heads across two sequential message-passing layers. This bounded depth prevents the over-smoothing of node features while still allowing the network to aggregate vital context from a node’s local topology.

Through this mechanism, the encoder intrinsically learns the semantic value of various edge types—for example, mathematically distinguishing between a standard `GenericWrite` and a high-privilege `MemberOf` relationship. The final output of the encoder is a dense, concatenated embedding matrix where every object in the `AD` network is represented as a sophisticated 64-dimensional vector that encapsulates its structural privilege and potential attack utility.

2.3.2 Policy Head

Following the extraction of topological features, the navigation model must evaluate and rank potential escalation paths. This decision-making phase is governed by the Policy Head. During traversal, the inference algorithm isolates the attacker’s current

node and identifies all topologically valid outbound neighbours.

To score these candidate transitions, the Policy Head leverages the embeddings generated by the **HGT** encoder. Rather than computing a rigid concatenation of the current state and a fixed target state, the model evaluates the transition via the Hadamard product (element-wise multiplication) between the current node’s expanded embedding and each candidate neighbour’s embedding. This element-wise interaction matrix is then processed through a **Multi-Layer Perceptron (MLP)** consisting of consecutive linear transformations, **Rectified Linear Unit (ReLU)** activations, and dropout layers.

The **MLP** ultimately compresses this latent relationship into a single scalar confidence score for each candidate. By relying on the relative, multiplicative interaction between node embeddings, the policy head acts as a generalised privilege evaluator. It steers the algorithmic traversal toward high-value nodes based strictly on the structural context learned during the encoding phase, completely decoupling the pathfinding logic from hardcoded heuristic rules.

2.3.3 Loss Function and Step Cost

To train the policy head to navigate not just efficiently, but stealthily, the architecture employs a weighted cross-entropy loss function. The effective training weight for each transition is calculated as follows:

$$\text{eff_weight} = \text{path_weight} \times (1 - \text{step_cost}) \quad (2.1)$$

where **path_weight** reflects the overall visibility and quality of the full attack path, and **step_cost** penalises individual transitions according to their noise category in Table 2.1. A high **step_cost** reduces the effective training weight of that transition, making the model less likely to replicate it when a lower-cost alternative exists in the training data. Conversely, topological transitions with a low **step_cost** receive a higher effective weight and are reinforced more strongly during training. The result is a model that, when faced with a choice between two structurally valid transitions, gravitates toward the one with less environmental impact — not because it was explicitly told to, but because the training signal made that choice more frequent and more rewarded.

This reward shaping mechanism fundamentally alters the pathfinding paradigm compared to traditional graph traversal algorithms. Standard shortest-path algorithms, such as Dijkstra’s algorithm implemented in default BloodHound, traverse the network by treating all edges as having an equal weight of one. Consequently, they inherently optimise for the shortest structural distance, which frequently results in highly detectable, noisy exploit chains.

By integrating the **step_cost** directly into the loss function, the neural network is mathematically discouraged from selecting loud transitions during the imitation learning

phase. Instead of merely memorising the shortest routes from the training corpus, the policy head internalises an OPSEC-driven heuristic. During inference, this enables the model to discover non-linear, multi-hop [Living of the Land \(LotL\)](#) paths that exhibit a significantly lower cumulative detectability cost than routes generated by unweighted Dijkstra searches on the exact same topology. This capability to prioritise stealth over topological proximity forms the core justification for deploying deep learning in automated offensive navigation.

2.4 Beam Search Navigation

2.4.1 Beam Search Navigation

Algorithm

The core pathfinding engine during inference is a heuristic beam search algorithm designed to evaluate and navigate the topological structures extracted by the [HGT](#) encoder. At each transition step, the algorithm queries the policy head to score all valid outbound neighbours, converting the raw logits into a probability distribution via a Softmax function. Rather than selecting the single best transition, the beam search maintains a tracked list of the top k candidate paths (the beam width), expanding each and accumulating their transition probabilities logarithmically to prevent mathematical underflow.

Crucially, the algorithm includes a fallback mechanism for undocumented or [Out Of Vocabulary \(OOV\)](#) edge types. If the inference engine encounters a relationship that the neural network did not observe during training, it bypasses the policy head’s probabilistic output and assigns a static baseline operational score of 0.05. This ensures the pipeline remains resilient and does not crash when deployed against evolving [AD](#) environments with novel schema additions.

Algorithm 1: Heuristic Beam Search for Attack Path Discovery**Input:** Graph G , Start Node S , Target Node T , Beam Width k , Max Depth D **Output:** Top k Attack Paths

```

1 beam  $\leftarrow$  [(score: 0.0, path: [ $S$ ])]
2 completed  $\leftarrow$  []
3 for  $d \leftarrow 1$  to  $D$  do
4   candidates  $\leftarrow$  []
5   foreach (score, path)  $\in$  beam do
6      $C \leftarrow$  path[-1]
7     if  $C = T$  or HasDCSync( $C, T$ ) then
8       completed.append((score, path))
9       continue
10    end
11    neighbours  $\leftarrow$  GetValidNeighbours( $C$ )
12    probs  $\leftarrow$  PolicyHead( $C$ , neighbours)
13    foreach  $N, prob \in$  (neighbours, probs) do
14      if IsOOVEdge( $C, N$ ) then
15        prob  $\leftarrow$  0.05
16      end
17      new_score  $\leftarrow$  score + log(prob)
18      candidates.append((new_score, path + [ $N$ ]))
19    end
20  end
21  beam  $\leftarrow$  TopK(candidates,  $k$ )
22  if length(completed)  $\geq k$  then
23    break
24  end
25 end
26 return TopK(completed  $\cup$  beam,  $k$ )

```

Terminal State: *Directory Replication Service synchronisation attack (DCSync)*

To optimise traversal and mimic the operational objectives of a professional red team, the inference engine employs a dynamic, state-aware termination condition rather than blindly searching for an exact target node match. At each step, the algorithm evaluates whether the current node possesses **DCSync** privileges over the target's domain. Specifically, it checks for the combination of **GetChanges**, **GetChangesAll**, and **GetChangesInFilteredSet** directory replication rights.

If a path reaches a node holding these privileges, the algorithm recognises global reachability and immediately halts, trimming any subsequent expansion. This assumes total domain compromise, acknowledging that an attacker with [DCSync](#) rights can organically forge the remaining transitions to reach the final objective without requiring further algorithmic guidance.

Cross-Environment Inference and Global Coordinate System

A critical engineering challenge during deployment is bridging the gap between the strictly heterogeneous data structures required by the neural network and the homogeneous, indexed lookups required by classical search algorithms.

During the initial ingestion and encoding phase, the graph strictly maintains its heterogeneity. The [HGT](#) processes the environment using isolated feature dictionaries and type-specific edge indices, ensuring the message passing mathematically preserves the distinct semantic nature of users, computers, and disparate [AD](#) permissions.

However, once the latent embeddings are extracted, tracking complex multidimensional tuples (e.g., ('computers', 12)) during a rapid, multi-hop heuristic beam search introduces severe computational overhead. To resolve this, the inference module dynamically constructs a unified global coordinate system post-encoding.

The heterogeneous embeddings are concatenated into a singular, flattened tensor space, and all structural edges are mathematically shifted using pre-calculated type offsets. This creates a global adjacency mapping for the deterministic search algorithm. It allows the beam search to execute highly efficient integer-based traversals (e.g., jumping from global Node 15 to Node 102) while still querying the policy head with the rich, heterogeneously derived latent vectors. This dynamic ingestion and coordinate resolution allow the model to execute inferences on completely unseen, enterprise-scale forests without requiring any structural modifications to the base [HGT](#).

2.4.2 Audit Module

Because deep learning models operate probabilistically, the pipeline includes a deterministic audit module to evaluate the semantic quality and operational viability of the generated attack paths. The module categorises outputs into three distinct reliability tiers:

- **Ok:** The path successfully reaches the explicit target node or terminates at a node possessing direct [DCSync](#) privileges over the target domain, signifying a complete operational success.
- **Warn:** The path identifies a critical escalation route but requires human improvisation to complete. For example, this occurs if the model achieves [DCSync](#) on a

foreign domain, necessitating a cross-forest SID History attack, or if the beam search terminates structurally with noisy edges such as `MemberOf` or `Contains` without reaching the final target.

- **Fail:** The path represents an operational dead-end. This is triggered if the beam search is unable to progress past the starting node or terminates at an unprivileged object lacking domain replication rights.

By pairing neural pathfinding with this strict heuristic audit, the tool ensures operators receive transparent, actionable intelligence alongside clear diagnostics when the algorithmic routing falls short.

2.5 Validation Module & Logging

To ensure operational transparency and deterministic execution, the architecture implements a centralised logging framework that captures both standard output and error streams across all pipeline stages. From the initial heterogeneous data ingestion to the final algorithmic traversal, every data mutation, structural shift, and inference decision is immutably recorded. By multiplexing the execution stream—providing operators with real-time diagnostic feedback while simultaneously committing telemetry to a persistent log—the system guarantees a comprehensive historical audit trail for subsequent evaluation or architectural debugging.

Prior to initiating the neural network or training phases, a dedicated validation module systematically audits the extracted graph tensors and the supervised training corpus against a strict set of expected baselines. It evaluates the mathematical health of the embeddings, confirms the preservation of critical operational pathways (such as [ADCS](#) escalation routes), and eliminates topological anomalies like self-loops. Crucially, it cross-references the expected outcomes—the documented target hops within the training examples—against the constructed graph to ensure all objective coordinates accurately map to valid boundaries within the heterogeneous tensors. By classifying structural deviations as either informational warnings or fatal execution blockers, the module guarantees that only mathematically sound and topologically viable data reaches the [HGT](#) encoder.

Chapter 3

Implementation

3.1 Technical Stack

The **GNN-AD-Navigator** pipeline is engineered entirely in Python, leveraging an ecosystem of specialised libraries for deep learning, graph processing, and interactive visualisation. The core neural network architecture and heterogeneous tensor operations are implemented using **PyTorch** and **PyTorch Geometric (PyG)**. For graph topology manipulation and deterministic traversal algorithms (such as the heuristic beam search), the system relies heavily on **NetworkX**.

To provide operators with an accessible and interactive frontend, the audit module and visualisations are built using **Streamlit**, coupled with **Pyvis** for dynamic, browser-based rendering of the discovered attack paths.

Model training was offloaded to cloud-based Kaggle compute instances to manage the heavy matrix multiplications required by the Heterogeneous Graph Transformer. Specifically, the training pipeline was pinned to dual NVIDIA Tesla T4 **Graphical Processing Units (GPUs)**. This specific hardware selection was a deliberate engineering constraint; older architectures like the Tesla P100 (which utilise the `sm_60` compute capability) exhibit architectural incompatibilities with the compiled CUDA binaries of modern **PyTorch** releases.

To ensure strict reproducibility across environments, the core dependencies were version-pinned as detailed in Table 3.1.

Library / Framework	Version
Python	3.10.x
PyTorch	2.1.0+cu118
PyTorch Geometric (PyG)	2.4.0
NetworkX	3.1
Streamlit	1.28.0
Pyvis	0.3.2

Table 3.1: Core technical stack and version pinning for environment reproducibility.

3.2 Repository Structure

The project repository is organised into a highly modular, pipeline-driven architecture. This separation of concerns ensures that data ingestion, neural network training, and frontend visualisation remain strictly decoupled. The directory tree is structured as follows:

```
gnn-ad-navigator/
├── examples/           # Sample AD graphs and expected pipeline outputs
├── gui/                # Streamlit frontend (app.py, visualisation.py)
├── lib/                # Core modules and helper functions
├── models/             # Serialised PyTorch weights (.pt checkpoints)
├── notebooks/          # Jupyter/Kaggle notebooks for model training
├── scripts/            # Core pipeline execution scripts
│   ├── build_dataset.py
│   ├── clean_bloodhound.py
│   ├── inference.py
│   ├── merger.py
│   ├── prepare_training_examples.py
│   ├── stitcher.py
│   └── validate_dataset.py
├── thesis/             # LaTeX source code for the manuscript
├── .dockerfile          # Containerised deployment configuration
├── .gitignore           # Strict ignore rules for sensitive data
├── launch.sh           # Master bash script for pipeline execution
├── README.md           # Documentation and setup instructions
├── requirements.txt     # Python dependencies
└── setup.sh            # Environment initialisation script
```

A critical security consideration within the repository’s configuration is the `.gitignore` file. Given the highly sensitive nature of Active Directory environments, the repository enforces strict exclusion paranoia. All `.json` scans, `Certipy` outputs, and compiled `.pt` tensors are explicitly ignored by version control to ensure that no real-world or client-specific infrastructure data is ever inadvertently committed to the public history.

3.3 Data Pipeline

3.3.1 Script Responsibilities and Data Flow

[TODO: Write pipeline data flow section — include input/output table]

3.3.2 SharpHound Version Normalisation

[TODO: Write SharpHound normalisation section]

3.4 Training

3.4.1 Training Data: GOAD

[TODO: Write GOAD training data section]

3.4.2 Kaggle Training Notebook

[TODO: Write Kaggle notebook section]

3.4.3 Model Checkpoints

[TODO: Write model checkpoints section]

3.5 Graphical & Command Line Interface

3.5.1 CLI

Commands and interactions

3.5.2 GUI

Design and Theme

[TODO: Write GUI design section]

Workflow Integration

[TODO: Write GUI workflow section]

Visualisation

[TODO: Write GUI visualisation section]

Chapter 4

Experiments and Results

4.1 Experimental Setup

4.1.1 Test Environment: INLANEFREIGHT

INLANEFREIGHT is an enterprise Active Directory environment provided by HTB Academy as part of their penetration testing learning paths. Unlike GOAD, INLANEFREIGHT was never used during training making the results obtained on it a zero-shot generalisation. The environment spans multiple domains connected through cross-domain trust relationships, with a total of approximately 4000 nodes, significantly larger and more complex than the training environment.

Data collection was performed using `bloodhound-python` for both testing and training environments, which produced the BloodHound JSON files later processed by the pipeline. ADCS data was collected separately using Certipy and injected into the graph at the stitching stage. It is worth noting that `bloodhound-python` has known collection limitations compared to SharpHound C#, particularly around cross-forest foreign group membership, a constraint that directly affected one of the queries evaluated in Section 4.5.

4.1.2 Diagnostic Environment: GOAD

While INLANEFREIGHT provides a realistic enterprise environment offers only for one targeted documented path to compromise, same goes for the child domain LOGISTICS.INLANEFREIGHT.LOCAL and the same forest trusted domain FREIGHT-LOGISTICS.LOCAL each of them with a specific cross domain attack path. The domain works well for generalisation validation but it is insufficient for dissecting model behaviour in depth.

GOAD, despite being the training environment, serves a second purpose here as a diagnostic environment. Its multi-domain structure and richer set of documented attack paths make it better suited for probing the model's limitations, identifying where it fails and why. Queries run against GOAD in this context are not used to measure generalisation (the model has seen this environment) but rather to isolate

specific architectural weaknesses under controlled conditions. Worth noting is that the path mentioned later are not provided in the training data.

4.1.3 Evaluation and Limitation Identification Logic

The evaluation methodology follows a continuous prompting approach: the model is repeatedly queried across different start and target node pairs, and its output is compared against BloodHound’s built-in pathfinding on the same graph. The choices the model makes at each step, combined with knowledge of the pipeline’s collected data and the `bloodhound-python` injector’s known limitations, allow for precise isolation of each failure mode and a confident proposal of remediation actions for future work.

Queries are not filtered to show only successes. Failed queries, degenerate paths, and cases where the model’s output diverges significantly from BloodHound’s are reported alongside valid results. This approach avoids a cherry-picking reading of the results: the model is evaluated at its coverage boundary, and failures are treated as diagnostic data.

4.2 Result 1: `wley` → Domain Admin on INLANEFREIGHT

Query and Context

To establish a baseline for the model’s navigational capabilities, the initial query targets a well-documented exploit chain within the INLANEFREIGHT enterprise environment. The objective is to navigate from the unprivileged user `wley` to achieve Domain Admin privileges. This scenario serves as a critical competency test: evaluating whether the model can apply sequential privilege escalation mechanics learned in a small, 351-node laboratory (GOAD) to a structurally complex, unseen 4 000-node enterprise graph.

Recovered Path

Operating purely on its learned policy with an HGT confidence score of 0.5620, the tool correctly provides the documented path. Navigates from the unprivileged user `wley` to reach the `domain admins` group. The tool given a new unseen Active Directory environment, navigates and recommends the same documented path, traversing edges the same way it had learned during training on this new architecture.

1. **Password Reset:** `wley` executes a `ForceChangePassword` attack on the user `damundsen`.
2. **Targeted Write:** `damundsen` abuses `GenericWrite` privileges to compromise the `help desk level 1` group.

3. **Group Inheritance:** This access provides an implicit `MemberOf` transition into the information technology group.
4. **Control DCSync user:** The information technology group exercises `GenericAll` over the user `adunn`.
5. **Domain compromise:** The user `adunn` is connected via the ACL edge `GetChangesAll` to the domain controller, allowing it to dump the `NTDS.nit` database file.

Crucially, the pipeline’s audit module halts the beam search precisely at `adunn`. It verifies that this user natively holds the required rights (`GetChanges` and `GetChangesAll`) to execute a DCSync attack on the target domain, terminating the search with confirmed operational success.

```

1
2 Path 1:
3 Score: 0.5620 Steps: 4
4 [1] wley@inlanefreight.local
5     -> damundsen@inlanefreight.local
6     edge    : forcechangepassword
7     attack  : Password reset
8     action  : Force-change target's password
9 [2] damundsen@inlanefreight.local
10    -> help desk level 1@inlanefreight.local
11    edge    : genericwrite
12    attack  : ACL abuse - targeted write
13    action  : Modify SPN, msDS-
14              AllowedToActOnBehalfOfOtherIdentity
15 [3] help desk level 1@inlanefreight.local
16    -> information technology@inlanefreight.local
17    edge    : memberof
18    attack  : Group inheritance
19    action  : Inherit rights via group membership
20 [4] information technology@inlanefreight.local
21    -> adunn@inlanefreight.local
22    edge    : genericall
23    attack  : ACL abuse - full control
24    action  : Reset password, add SPN, modify object
25
26 AUDIT:
    Path terminates at a node with DCSync on the target domain
    'inlanefreight.local'. This is operational success.

```

Listing 4.1: model output for escalation path on INLANEFREIGHT

4.3 Result 2: Cross-Domain Trust Traversal

```
1 Start : htb-student_adm@logistics.inlanefreight.local (users)
2 Target : domain admins@inlanefreight.local (groups)
3
4
5 Path 1:
6   Score: 0.2349 Steps: 6
7   [1] htb-student_adm@logistics.inlanefreight.local
8       -> domain admins@logistics.inlanefreight.local
9       edge : memberof
10      attack : Group inheritance
11      action : Inherit rights via group membership
12   [2] domain admins@logistics.inlanefreight.local
13       -> logistics.inlanefreight.local
14       edge : writedacl
15      attack : ACL abuse - DACL modification
16      action : Grant yourself GenericAll on target
17   [3] logistics.inlanefreight.local
18       -> inlanefreight.local
19       edge : sameforesttrust
20      attack : Same-forest trust traversal
21      action : Cross domain via SID History
22   [4] inlanefreight.local
23       -> freightlogistics.local
24       edge : crossforesttrust
25      attack : Cross-forest trust traversal
26      action : Inter-forest pivot
27   [5] freightlogistics.local
28       -> domain controllers@freightlogistics.local
29       edge : contains
30      attack : Container hierarchy
31      action : Object lives inside OU/container
32   [6] domain controllers@freightlogistics.local
33       -> academy-ea-dc03.freightlogistics.local
34       edge : contains
35      attack : Container hierarchy
36      action : Object lives inside OU/container
37
38 AUDIT:
```

```

39      X Path terminates at 'academy-ea-dc03.freightlogistics.
      local' which has no DCSync rights
40      and is not the target. The model could not find a working
      attack chain.
41      ! Final node is in domain 'academy-ea-dc03.freightlogistics
      .local' but target is
42      in 'inlanefreight.local'. Cross-domain attack may need
      additional steps.

```

Listing 4.2: model output for cross-domain trust traversal on INLANEFREIGHT

The model navigated the cross domain trust edge, recommending the correct documented path. The processed graph portrays correctly the trust between domains. The model identified the edge type seen during training and traverses it. The rest of the path is noise a direct result of an architectural limitation. The specified target node `admins@inlanefreight.local` does not have a direct incoming path. When all the nodes are outgoing the model panics and produces noise. The audit script catches the errors and alerts the user.

4.3.1 Cross-Forest Foreign MemberOf

```

1
2 Start  : administrator@inlanefreight.local (users)
3 Target : administrators@freightlogistics.local (groups)
4
5 Path 1:
6   Score: 0.0918  Steps: 1
7   [1] administrator@inlanefreight.local
8       -> administrators@freightlogistics.local
9       edge      : memberof
10      attack    : Group inheritance
11      action    : Inherit rights via group membership
12
13 AUDIT:
14   Path reaches the requested target directly.

```

Listing 4.3: Model output on unseen node

One common misconfiguration in Active Directory forests, when the team with admin rights responsible for Active directory and network configurations, gives the admins of two domains membership of the same group. The admin account of one domain now holds admin rights over the other domain, allowing for easier interventions for the admin to adjust any configuration within the other domain in the same forest.

The pipeline manages to extract the `cross-forest Foreign MemberOf` edge presenting it to the model. Once seen during training, the tool outputs the correct ACL abuse path to the user.

4.4 Result 3: Constrained Delegation

```

1 Start   : castelblack.north.sevenkingdoms.local (computers)
2 Target  : kingslanding.sevenkingdoms.local (computers)
3
4 Path 1:
5     Score: 1.0000 Steps: 2
6     [1] castelblack.north.sevenkingdoms.local
7         -> winterfell.north.sevenkingdoms.local
8         edge    : allowedtodelegate
9         attack   : Constrained Delegation
10        action  : S4U2Self + S4U2Proxy impersonation
11     [2] winterfell.north.sevenkingdoms.local
12        -> enterprise domain controllers@sevenkingdoms.local
13        edge    : memberof
14        attack   : Group inheritance
15        action  : Inherit rights via group membership
16
17 AUDIT:
18     Path terminates at a node with DCSync on the target domain
        'kingslanding.sevenkingdoms.local'. This is operational
        success.

```

Listing 4.4: Model output on unseen node

Key insight to foreground, the `AllowedToDelegate` edge was missed during the graph construction during training. The `AllowedToDelegate` is an edge type not directly present in the bloodhound scan, it needed further processing deducing the relationship between the two edges before establishing its existence. Hence during training, the model did not see this edge type. When introduced for this query as the only navigable node, the model suggested traversing it and gave a score of 1 (the only probable path).

4.5 Result 4: Documented Limitations

castelblack → *braavos.essos.local*

This query produces an interesting divergence between the model’s output and BloodHound’s pathfinding. The model navigates: *castelblack* → *winterfell* (*AllowedToDelegate*) → *enterprise domain controllers@sevenkingdoms* (*MemberOf*) → *sevenkingdoms.local* (*GetChanges*, partial *DCSync*) → *essos.local* (*CrossForestTrust*) → *laps@essos* (*Contains*) → *braavos.essos.local* (*Contains*).

BloodHound identifies a structurally different route passing through *CoerceToTGT* on *north.sevenkingdoms.local*, followed by a *SameForestTrust* pivot and an ACL chain through *tyron.lannister* (*GenericWrite*) and *ReadLAPSPassword* toward the target.

The divergence is explained by two limitations working together. First, *CoerceToTGT* is a protocol-level coercion primitive invisible to the pipeline and absent from the training data entirely, the model has no learned signal for this edge type and cannot reproduce BloodHound’s route. Second, the two *Contains* hops at the end of the model’s path are structural noise: the model reaches *essos.local* via a valid *CrossForestTrust* pivot, gets the *DCSync* rights but continues walking into container hierarchy rather than stopping or instantly recommending *Braavos* as the immediate final target. This behaviour reflects the terminal state blindness discussed in Section ??.

Neither path is strictly superior. The model’s route relies on a partial *DCSync* and a cross-forest trust traversal its a valid operational chain. BloodHound’s route relies on *CoerceToTGT*, requiring active protocol-level interaction beyond what static graph analysis can represent.

4.6 Discussion

4.6.1 Model Coverage and Training-Set Ceiling

The training environment GOAD, allows for about 10 documented paths, each tested and reproduced before adding to the csv file. The extracted paths are then evaluated based on visibility and how much topologically dependant each of their transitions are. In total the final dataset is about 49 different transition. Filtering the invisible and non learnable attacks, leaves us with 33 viable and trainable attacks. This final number directly influences the architecture of the system. The first approach was to have a context aware policy head, that stores the user creds collected during attack paths, it was meant to have the model navigate and base its future suggested paths on the user inventory. This methodology required an extended dataset and Crucially time that i am limited on. Later, changed it to only having the model understand what creds where

used with what techniques but at this scale, most of the documented viable attacks relied on the current node’s creds giving no new information to the model.

Attack inventory generation is a natural extension but requires a different training objective, a larger dataset and is left as future work.

[TODO: Write coverage discussion]

4.6.2 Expressiveness vs. Transferability: HGT vs. GCN

During the diagnostic phase we observed the GCN baseline alongside BloodHound’s standard shortest-path outputs. From extensive querying, the GCN appears to produce paths that sit somewhere between a naive shortest path and a fully stealthy route. However, we noticed that it frequently misses direct hits on the target. Because the GCN treats all connections equally, this lack of edge-based attention seems to hinder its ability to directly abuse specific misconfigurations. It appears to struggle to distinguish between a highly detectable attack edge and a quiet structural link.

On the other hand, the HGT computes attention differently for each type of relationship. This allows the HGT to capture the practical meaning of different Active Directory edges, recognizing that an ALLOWEDTODELEGATE edge is much quieter than a forced password reset. In our tests on the training data (GOAD), this expressiveness helped the HGT perform very well, as it learned to select precise and stealthy exploit chains.

However, when evaluating the models on the unseen 4000-node INLANEFREIGHT environment, we noted an interesting trade-off. The GCN actually produced slightly higher raw confidence scores than the HGT ($GCN \geq HGT$). This observation suggests that the HGT’s complex attention mechanism might be slightly overfitting to the specific graph patterns it saw during training. Meanwhile, the simpler design of the GCN seems to transfer more smoothly to new environments, since it relies on general node closeness rather than exact sequences of edge types.

Ultimately, both models successfully recovered the correct attack paths. The difference we observed was purely in their raw probability scores, rather than their actual ability to find a working route.

[TODO: Write HGT vs. GCN discussion]

4.6.3 Pipeline Completeness impact

One of the most interesting observations from Result 3 the Constrained Delegation path is how the model handles completely unseen edge types. During training on the GOAD dataset, the model never encountered an ALLOWEDTODELEGATE edge. However, once we updated the data pipeline to extract this specific edge type, the model successfully traversed it during inference without any retraining.

This behavior suggests that the model’s ability to discover new attack paths is heavily tied to the completeness of the data collection pipeline. Because the model uses the graph’s physical topology to evaluate possible next steps, introducing a new edge type simply adds a new structural pathway. Even without learned attention weights specifically tuned for `ALLOWEDToDELEGATE`, the model still opted to traverse it because it represented the most viable route to progress toward the target.

This decoupling of graph coverage from model capability is a significant takeaway. It implies that we might not need to retrain the neural network from scratch every time a new Active Directory vulnerability or edge type is discovered. Instead, simply updating the extraction tools (like `build_dataset.py`) to map the new edge seems sufficient to immediately expand the model’s operational reach.

For this reason, we consider the completeness and accuracy of the data pipeline to be a major contribution, playing a role that is just as critical as the neural network architecture itself.

Chapter 5

Limitations and Future Work

While the empirical evaluation in Chapter 4 demonstrates that the Heterogeneous Graph Transformer [HGT](#) successfully learns and generalises offensive navigation policies, the current pipeline remains a proof of concept. Transitioning from a controlled laboratory environment to a production-ready red teaming tool requires an honest assessment of the system’s boundaries. This chapter systematically details the fundamental mathematical, algorithmic, and operational limitations of the current approach, and proposes a concrete engineering roadmap for a second-generation architecture.

5.1 Mathematical and Algorithmic Limitations

The most significant constraints on the GNN-AD-Navigator do not stem from data ingestion or environmental scanning, but from the inherent mechanics of deep learning when applied to complex, asymmetrical graph structures. Understanding these mathematical flaws is critical for interpreting the model’s behaviour during deep attack simulations.

5.1.1 Multiplicative Probability Decay and Search Bias

The inference phase relies on a heuristic beam search to navigate the Active Directory graph. At each step, the policy head outputs a Softmax probability distribution over all available candidate edges. The total confidence score for a given attack path is then calculated by multiplying the individual probabilities of each hop along that route.

Mathematically, multiplying fractions consistently yields a smaller product (e.g., $0.8 \times 0.8 \times 0.8 = 0.512$). In the context of Active Directory attack paths, this creates a phenomenon known as *multiplicative probability decay*. The search algorithm inherently penalises long attack chains simply because they require more mathematical transitions.

From an offensive security perspective, this mathematical decay directly contradicts the foundational objective of the navigation policy. The primary motivation for engineering transition-specific confidence scores—effectively acting as an [Operational Security \(OPSEC\)](#) step cost—was to explicitly train the model to prioritise stealth over path efficiency. In professional red teaming engagements, operators heavily rely on

LotL techniques, deliberately chaining multiple low-privilege, native Active Directory misconfigurations to escalate privileges without triggering [Security Information and Event Management \(SIEM\)](#) (Security Information and Event Management) alarms. The model was designed to recognise these stealthy, deep attack chains as operationally superior to a noisy, direct exploit. However, because the probability decay compounds with every additional transition, the mathematical penalty of path length eventually overwhelms the learned OPSEC rewards. As a result, the beam search prematurely prunes the desired stealthy chains, paradoxically forcing the model to revert to the exact behaviour it was engineered to avoid: demonstrating a strong preference for shorter, highly detectable attacks simply because they require fewer algorithmic multiplications.

5.1.2 Bounded Receptive Field and the Horizon Problem

The expressiveness of a Graph Neural Network is fundamentally tied to its receptive field, which is dictated by the number of hidden layers (K). During the message-passing phase, a node aggregates feature data from K hops away. Therefore, to intrinsically understand the structural relationship between a compromised starting user and a target Domain Controller seven steps away, the model would theoretically require seven layers of message passing.

However, Active Directory environments are highly dense, "small world" networks, characterised by heavily nested group memberships and extensive permissions. In such dense topologies, expanding the depth of the network triggers a well-documented graph learning limitation known as *over-smoothing*. If the Heterogeneous Graph Transformer [HGT](#) is configured with too many layers, the repeated aggregation of neighbourhood features causes the embeddings of structurally distinct nodes—such as a standard workstation and an Enterprise Admin group—to mathematically converge toward indistinguishable mean vectors.

To preserve the distinct privilege hierarchies required for accurate routing, the GNN-AD-Navigator's architecture is strictly bounded to a shallow depth (specifically $K = 2$ layers, as established in the methodology). While this successfully prevents over-smoothing and maintains structural fidelity, it directly introduces the *horizon problem*. When executing a deep attack simulation (e.g., a 7-hop path), the starting node's embedding contains zero latent information about a target that lies beyond its two-hop receptive field. The policy head is effectively blind to the distant destination, forcing the algorithm to navigate using only local privilege gradients rather than true, end-to-end topological awareness.

5.1.3 The "Blind Compass" Effect in Lateral Movement

As detailed in Chapter 2, a critical engineering adaptation within the pipeline was the design of the policy head. Rather than computing a target-aware concatenation, the model utilises the Hadamard product (element-wise multiplication) between the current node's embedding and the candidate neighbour's embedding. This architectural choice proved highly effective for the primary goal of Active Directory exploitation: privilege escalation. By forcing the model to evaluate the absolute semantic value of a transition, it successfully learned a generalised privilege hierarchy. It acts as a "universal climber" that inherently gravitates toward high-value nodes like Domain Admins, regardless of the specific topology.

However, this adaptation introduces a distinct operational trade-off. Because the target node's embedding is mathematically excluded from the transition scoring, the policy head possesses no "target gravity." The model implicitly knows how to escalate privileges, but it does not inherently know its final destination.

This limitation becomes acutely apparent during horizontal lateral movement. If an attack simulation requires navigating through a flat network segment—such as pivoting between multiple standard workstations within an isolated Organizational Unit (OU)—the model struggles to find an optimal route. When presented with candidate neighbours that share identical, low-privilege weights, the policy head lacks the directional context required to efficiently break the mathematical tie. Without the target vector actively pulling the algorithm toward a specific coordinate, the search can oscillate or wander inefficiently. The model effectively acts as a "blind compass"—perfectly calibrated to point "up" toward domain dominance, but unable to navigate laterally toward a specific, non-administrative target.

5.2 Operational and Data Boundaries

A fundamental axiom of machine learning is that a model's operational capacity is strictly bounded by the fidelity of its underlying data. While the algorithmic limitations define how the model navigates the graph, the operational boundaries dictate whether the necessary topological pathways exist in the underlying dataset to begin with.

5.2.1 Edge Directionality and the Computer Node Sinkhole

During the empirical evaluation, the model occasionally exhibited suboptimal routing by bypassing topologically logical hops through specific Computer nodes. This behaviour is not an algorithmic failure, but rather a direct consequence of how `PyTorch Geometric` enforces strict edge directionality, combined with the inherent nature of Active Directory Access Control Lists (ACLs).

In a standard BloodHound-compatible graph, privileges are mapped as directed edges from a principal (the controller) to a target (the controlled). Consequently, Computer nodes predominantly accumulate incoming edges (e.g., Users or Groups holding `GenericAll` or `AdminTo` rights over the machine) but possess virtually zero outgoing rights over other objects in the domain. Without the ability to dynamically resolve and traverse active user sessions (e.g., using a `HasSession` edge to pivot from the compromised machine to a highly privileged Domain Admin logged into it), the Computer node becomes a topological sinkhole.

The inference engine mathematically learns to avoid these nodes because traversing into them almost always results in a dead end. While a human operator would compromise the computer, dump memory credentials (e.g., via `Mimikatz`), and pivot, the current graph schema does not explicitly represent that post-exploitation capability as a traversable outgoing edge.

5.2.2 Collector Incompleteness and Schema Evolution

The current data pipeline relies heavily on the Linux-compatible Python ingestor (`bloodhound-ce-python`). While this ensures the toolset remains accessible from standard UNIX-based attack hosts, it introduces severe structural blind spots. Most notably, the Python collector exhibits incompleteness regarding the enumeration of explicit Trust accounts. Furthermore, it lacks native support for Active Directory Certificate Services (ADCS), necessitating reliance on legacy versions of `Certipy` and complex logic to stitch the graph together.

This fragility was further exacerbated by environmental volatility during graph enumeration phases. For example, hypervisor kernel incompatibilities and virtual network bridging failures during the provisioning of the `ESSOS.local` test domain demonstrated that relying on fragmented, multi-tool data collection is difficult to scale consistently.

Ultimately, this is an ingestion constraint, not a model limitation. For a production-grade deployment, future iterations must deprecate the Python ingestor in favour of the official C# SharpHound collector. Executing SharpHound natively within the target environment would inherently surface coercion-derived edges, complete trust mappings, and richer ACL enumeration, directly expanding the model’s discoverable attack surface without requiring any architectural modifications to the neural network.

5.2.3 ADCS Edge Sparsity and Static Stitching

The current dataset exhibits severe class imbalance regarding [ADCS](#) attack primitives. Within the training corpus, [ADCS](#) template transitions—such as `Enroll` or `WriteDacl` over a `CertTemplate` node—are extremely scarce, typically limited to only one or two

traversable instances per template.

From a deep learning perspective, this edge sparsity presents a significant challenge for the HGT. Neural networks require sufficient statistical frequency to accurately learn the latent semantic weight of a transition. Because the model observes thousands of `MemberOf` or `GenericAll` relationships but only a handful of `ADCS` escalations, the policy head inherently struggles to assign an operationally confident probability score to certificate-based attacks during inference, occasionally causing the beam search to undervalue them.

Furthermore, this sparsity is compounded by the pipeline’s ingestion mechanics. Because the Python collector natively lack comprehensive `ADCS` enumeration, these specific edges are currently extracted via fragmented `Certipy` scans and statically stitched into the underlying graph using custom Python logic. While this static injection successfully bridges the topological gap and physically allows the model to traverse the `ADCS` infrastructure, the hardcoded nature of the stitching process remains brittle. Future iterations must address this data scarcity—either through the injection of synthetic `ADCS` traces to artificially amplify the training signal, or by adopting a unified, native collection tool that reliably surfaces certificate misconfigurations at scale.

5.3 Future Work: Algorithmic Upgrades

The mathematical constraints identified in Section 5.1 cannot be resolved simply by increasing the volume of training data or arbitrarily tuning hyperparameters. Addressing the horizon problem and the adverse effects of multiplicative probability decay requires fundamental architectural enhancements to the inference engine. The following algorithmic upgrades outline a clear engineering roadmap for a second-generation navigation tool, focusing on hybridizing deep learning embeddings with deterministic routing logic.

5.3.1 Waypoint Routing and Path Segmentation

To overcome the bounded receptive field of the Heterogeneous Graph Transformer, future iterations of the pipeline should implement a "divide-and-conquer" routing wrapper. As previously established, the model struggles with deep attack chains because targets beyond a two-hop radius fall completely outside the starting node’s latent awareness, creating a blind horizon.

A waypoint routing architecture would solve this by decoupling the global pathfinding strategy from the local transition predictions. Much like modern GPS navigation systems, a static pre-processor could analyse the global graph topology to identify major structural hubs or "choke points"—such as highly connected `OU`, Tier-0 infrastructure, or centralized identity providers.

Instead of querying the neural network to calculate a continuous, 10-hop path from a low-privileged user directly to a Domain Controller, the wrapper would segment the objective. The model would first be tasked with navigating to the nearest waypoint; upon success, the search state would reset and generate a new trajectory to the subsequent waypoint.

This segmentation directly neutralises both the horizon problem and the multiplicative probability decay. By ensuring that each individual path segment remains short, the intermediate targets stay within the model’s receptive field. Furthermore, resetting the probability calculation at each waypoint prevents the accumulated mathematical penalties from overwhelming the learned **OPSEC** step costs, allowing the model to freely select stealthy, non-linear routes.

5.3.2 Guided Search Heuristics (A^* Algorithm)

While waypoint routing mitigates the horizon problem, the fundamental inference mechanism must also evolve to protect OPSEC-driven routing. The current implementation relies on a heuristic beam search, which, as established, falls victim to multiplicative probability decay. To preserve long, stealthy attack chains, future architectures should replace the pure beam search with an additive, cost-based heuristic algorithm such as A^* (A-Star).

Unlike beam search, which evaluates routes by multiplying fractional Softmax probabilities, A^* evaluates candidate paths using an additive cost function: $f(n) = g(n) + h(n)$. In this paradigm, $g(n)$ represents the exact, accumulated OPSEC cost of the path taken so far, while $h(n)$ serves as the heuristic estimating the remaining distance or effort to the target. By summing step costs rather than multiplying probabilities, the mathematical penalty for path length grows linearly rather than exponentially.

This algorithmic shift would elegantly repurpose the existing deep learning architecture. Instead of acting as a standalone, greedy pathfinder, the **HGT** policy head would be deployed exclusively as the heuristic function, $h(n)$. The **GNN** would evaluate candidate neighbours and assign dynamic, context-aware proximity scores based on learned privilege hierarchies, while the deterministic A^* framework strictly tracks the accumulated stealth metrics ($g(n)$).

This hybrid approach ensures that operationally superior, low-privilege **LotL** chains are evaluated on their true tactical merit. It mathematically protects longer paths from being prematurely pruned, completely eradicating the current system’s bias toward short, highly detectable exploit paths.

5.3.3 Terminal State Learning and Global Reachability

The current inference engine relies on a hardcoded symbolic predicate to identify terminal states: if the search algorithm lands on a node possessing `DCSync` rights (e.g., `GetChanges` and `GetChangesAll` on the target domain), the search halts, assuming total domain compromise. While theoretically rudimentary, the empirical results from Chapter 4 demonstrate that this static cutoff condition is operationally sufficient and highly effective for current red teaming engagements.

A theoretically purer approach would involve replacing this hardcoded heuristic with Offline `Reinforcement Learning (RL)`. By training a dedicated value head over the existing traces using algorithms like Conservative Q-Learning (CQL), the model could intrinsically learn to assign massive reward values to `DCSync`-capable nodes. However, deploying an Offline RL architecture is exceptionally complex, computationally expensive, and prone to severe training instability. Given the high performance of the current static condition, introducing a brittle RL pipeline represents significant engineering overhead for diminishing returns.

A more pragmatic, data-driven alternative exists to teach the policy head the concept of global reachability without altering the underlying architecture. The training corpus could be augmented with synthetic traces where reaching a `DCSync`-capable node is immediately followed by a direct transition to the final target, regardless of whether a literal topological edge exists. By exposing the model to these synthetic "teleportation" transitions, the `HGT` would mathematically learn that owning a `DCSync` node equates to owning the entire graph. This allows the existing architecture to organically learn the semantic weight of domain dominance entirely through data representation, bypassing the need for a resource-heavy Reinforcement Learning overhaul.

5.3.4 LLM Integration for Out-Of-Vocabulary (OOV) Edges

As offensive enumeration tools like SharpHound evolve, their output schemas and edge vocabularies frequently change (e.g., introducing new attack primitives or altering nomenclature such as `WriteDACL` to `WriteDacl`). The current pipeline relies on a static alias mapping dictionary to normalise these relationships before tensor vectorisation. From an engineering perspective, this static approach is highly efficient, computationally lightweight, and operationally sufficient for the current Active Directory landscape.

However, maintaining a hardcoded vocabulary introduces long-term technical debt and fragility when encountering undocumented, `OOV` edges during a live engagement. A promising future direction is the integration of a localized, lightweight Large Language Model (LLM) to act as a semantic parser at the ingestion layer. Instead of dropping or ignoring unknown relationships, the LLM could semantically evaluate the new edge type and automatically map it to the closest known privilege vector in the model's latent

space. While static mappings are entirely sufficient for today’s red teaming operations, LLM-assisted parsing would provide a highly resilient, future-proof ingestion pipeline capable of organically adapting to evolving Active Directory schemas.

5.4 Conclusion

The limitations and proposed upgrades outlined in this chapter do not diminish the empirical success of the current **GNN-AD-Navigator**; rather, they define the precise engineering trajectory required to transition from a constrained proof-of-concept to an enterprise-grade autonomous agent.

An application that successfully surpasses these boundaries—coupling the deep semantic embeddings of a Heterogeneous Graph Transformer with the deterministic routing of an A^* heuristic and localised waypoint segmentation—would represent a paradigm shift in automated offensive security. Such a system would prove that AI-driven pathfinding can scale to massive, asymmetrical Active Directory environments without sacrificing the OPSEC requirements crucial for advanced **LotL** attack chains. It would effectively transition automated red teaming from rigid, shortest-path vulnerability scanners into dynamic, stealth-conditioned navigators.

More broadly, this research roadmap redefines the role of Graph Neural Networks in the cybersecurity domain. Historically relegated to defensive classification tasks, such as anomaly detection or alert clustering, this work establishes **GNNs** as foundational reasoning engines for offensive operations. By proving that neural networks can mathematically abstract and internalise the semantic hierarchy of privilege escalation, it lays the theoretical groundwork for a new generation of agentic AI—tools capable of navigating complex digital topologies with the intuitive, contextual awareness of a human Red Teamer.

General Conclusion

[TODO: Write general conclusion (1 page)]

Bibliography

- [1] HackTheBox academy. “Introduction to active directory.” (2023), [Online]. Available: <https://academy.hackthebox.com/module/74/section/1347> (visited on 06/10/2026).
- [2] Microsoft Corporation. “Active directory security architecture.” (2023), [Online]. Available: <https://learn.microsoft.com/en-us/windows-server/identity/> (visited on 06/21/2026).
- [3] C. Neuman, T. Yu, S. Hartman, and K. Raeburn, *The kerberos network authentication service (v5)*, Internet Requests for Comments, RFC, 2005.
- [4] W. Schroeder. “A guide to attacking domain trusts.” (2017), [Online]. Available: <https://www.harmj0y.net/blog/redteaming/a-guide-to-attacking-domain-trusts/> (visited on 06/22/2026).
- [5] W. Schroeder and L. Christensen, “An ace up the sleeve: Designing active directory dacl backdoors,” in *Black Hat USA*, Black Hat, 2017.
- [6] B. Delpy and V. L. Toux. “Mimikatz: Dcsync and the drsr protocol.” (2015), [Online]. Available: <https://github.com/gentilkiwi/mimikatz> (visited on 06/22/2026).
- [7] W. Schroeder and L. Christensen, “Certified pre-owned: Abusing active directory certificate services,” in *Black Hat USA*, Black Hat, 2021.
- [8] E. Shamir. “Wagging the dog: Abusing resource-based constrained delegation to attack active directory.” (2019), [Online]. Available: <https://shenaniganslabs.io/2019/01/28/Wagging-the-Dog.html> (visited on 06/22/2026).
- [9] SpecterOps. “Bloodhound: Six degrees of domain admin.” (2016), [Online]. Available: <https://github.com/BloodHoundAD/BloodHound> (visited on 06/22/2026).
- [10] O. Lyak. “Certipy-ad: Active directory certificate services enumeration and abuse.” (2022), [Online]. Available: <https://pypi.org/project/certipy-ad/> (visited on 06/24/2026).
- [11] D. jan Mollema. “Bloodhound-python: A python based ingestor for bloodhound.” (2018), [Online]. Available: <https://github.com/fox-it/bloodhound-python> (visited on 06/22/2026).

- [12] O. Lyak. “Certipy: Active directory certificate services enumeration and abuse.” (2022), [Online]. Available: <https://github.com/ly4k/Certipy> (visited on 06/23/2026).
- [13] T. Be’ery and T. Maor. “Gofetch: Automatically deploying privilege escalation tools.” (2017), [Online]. Available: <https://github.com/GoFetchAD/GoFetch> (visited on 06/22/2026).
- [14] V. L. Toux. “Pingcastle: Active directory security tool.” (2017), [Online]. Available: <https://www.pingcastle.com/> (visited on 06/22/2026).
- [15] MDPI, “Physics-informed graph neural networks for attack path prediction in active directory,” *Algorithms*, vol. 18, 2025. [Online]. Available: <https://www.mdpi.com/2624-800X/5/2/15>.
- [16] Z. Hu, Y. Dong, K. Wang, and Y. Sun, “Heterogeneous graph transformer,” in *Proceedings of The Web Conference 2020*, ACM, 2020, pp. 2704–2710.
- [17] SpecterOps. “Bloodhound community edition v8 launches with opengraph.” (2025), [Online]. Available: <https://specterops.io/blog/2025/07/29/bloodhound-community-edition-v8-launches-with-opengraph-identity-attack-paths-beyond-active-directory-entra-id/> (visited on 06/22/2026).
- [18] M. Boon. “Bloodhound on kali linux 2025: The guide i wish existed.” (2025), [Online]. Available: <https://medium.com/@mauriceboon88/bloodhound-plumhound-on-kali-linux-2025-the-guide-i-wish-existed-before-i-lost-2-hours-fighting-397e43035fa7> (visited on 06/22/2026).
- [19] X. Sun and J. Yang, “Hetglm: Lateral movement detection by discovering anomalous links with heterogeneous graph neural network,” in *2022 IEEE International Performance, Computing, and Communications Conference (IPCCC)*, IEEE, 2022. DOI: [10.1109/IPCCC55026.2022.9894347](https://doi.org/10.1109/IPCCC55026.2022.9894347).
- [20] Z. Hu, Y. Dong, K. Wang, and Y. Sun, “Heterogeneous graph transformer,” in *Proceedings of the Web Conference 2020*, 2020, pp. 2704–2710.

Appendix A

Sanitised BloodHound JSON Example

[TODO: Insert sanitised JSON excerpt as a lstlisting]

Appendix B

Heterogeneous Graph Schema

[TODO: Insert full graph schema table]

Appendix C

Key Code Excerpts

Effective Weight Loss Function

```
1 # eff_weight combines path importance with step discretion cost
2 eff_weight = path_weight * (1 - step_cost)
```

Listing C.1: Effective weight computation in `prepare_training_examples.py`

[TODO: Add beam search loop and `has_dcsync_on` excerpts here]

Appendix D

GUI Screenshots

[TODO: Insert GUI screenshots]